

```

adc1_channel1_checkbox.grid(column=1, row=1, padx=10, pady=5, sticky=W)

adc1_channel2_checkbox = ttk.Checkbutton(adc_frame, text="Channel 2", variable=adc1_channel2_var, width=checkboxwidth)
adc1_channel2_checkbox.grid(column=1, row=2, padx=10, pady=5, sticky=W)

adc1_channel3_checkbox = ttk.Checkbutton(adc_frame, text="Channel 3", variable=adc1_channel3_var, width=checkboxwidth)
adc1_channel3_checkbox.grid(column=1, row=3, padx=10, pady=5, sticky=W)

adc2_channel1_checkbox = ttk.Checkbutton(adc_frame, text="Channel 1", variable=adc2_channel1_var, width=checkboxwidth)
adc2_channel1_checkbox.grid(column=2, row=1, padx=10, pady=5, sticky=W)

adc2_channel2_checkbox = ttk.Checkbutton(adc_frame, text="Channel 2", variable=adc2_channel2_var, width=checkboxwidth)
adc2_channel2_checkbox.grid(column=2, row=2, padx=10, pady=5, sticky=W)

adc2_channel3_checkbox = ttk.Checkbutton(adc_frame, text="Channel 3", variable=adc2_channel3_var, width=checkboxwidth)
adc2_channel3_checkbox.grid(column=2, row=3, padx=10, pady=5, sticky=W)

adc3_channel1_checkbox = ttk.Checkbutton(adc_frame, text="Channel 1", variable=adc3_channel1_var, width=checkboxwidth)
adc3_channel1_checkbox.grid(column=3, row=1, padx=10, pady=5, sticky=W)

adc3_channel2_checkbox = ttk.Checkbutton(adc_frame, text="Channel 2", variable=adc3_channel2_var, width=checkboxwidth)
adc3_channel2_checkbox.grid(column=3, row=2, padx=10, pady=5, sticky=W)

adc3_channel3_checkbox = ttk.Checkbutton(adc_frame, text="Channel 3", variable=adc3_channel3_var, width=checkboxwidth)
adc3_channel3_checkbox.grid(column=3, row=3, padx=10, pady=5, sticky=W)

adc4_channel1_checkbox = ttk.Checkbutton(adc_frame, text="Channel 1", variable=adc4_channel1_var, width=checkboxwidth)
adc4_channel1_checkbox.grid(column=4, row=1, padx=10, pady=5, sticky=W)

adc4_channel2_checkbox = ttk.Checkbutton(adc_frame, text="Channel 2", variable=adc4_channel2_var, width=checkboxwidth)
adc4_channel2_checkbox.grid(column=4, row=2, padx=10, pady=5, sticky=W)

adc4_channel3_checkbox = ttk.Checkbutton(adc_frame, text="Channel 3", variable=adc4_channel3_var, width=checkboxwidth)

```



Mag. Julian
Kampitsch
HTBLA Graz-
Göding

AutoMeasure FleX

Teilbereich: GUI-Applikation zur Konfiguration eines Messaufbaus über
CAN-Bus

Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche kenntlich gemacht habe.

Graz, am 13. August 2026

Mag. Julian Kampitsch

Abstract (deutsch)

Die Diplomarbeit „AutoMeasure FleX – GUI-Applikation zur Konfiguration eines Messaufbaus über CAN-Bus“ beschreibt die Entwicklung einer grafischen Benutzeroberfläche zur Steuerung eines Messaufbaus und Kommunikation über den CAN-Bus. Ziel der Arbeit war es, eine funktionale, benutzerfreundliche und effiziente Anwendung zu realisieren, die eine flexible Konfiguration ermöglicht. Die Wahl der Programmiersprache und des Frameworks – Python 3.13 und Tkinter – wird eingangs diskutiert und damit begründet, dass dadurch eine schnelle Prototypentwicklung und einfache Wartung gewährleistet ist.

Die CAN-Bus-Kommunikation wurde auf Basis der PCAN-USB-API softwareseitig implementiert. Eine vollständige Integration und Validierung mit dem vorgesehenen Messaufbau konnte mangels Verfügbarkeit der Hardware im Projektzeitraum nicht durchgeführt werden. Zur Verbesserung der Softwarequalität wurden Programmierprinzipien wie PEP 8, Modularisierung und Fehlermanagement umgesetzt. Zudem wurde die Anwendung als ausführbare Datei ausgegeben, um eine reibungslose Nutzung ohne Python-Installation zu ermöglichen.

Nach erfolgreicher Umsetzung bleiben dennoch einige Aspekte für zukünftige Entwicklungen offen, insbesondere eine Datenvisualisierung oder die Optimierung der CAN-Bus-Kommunikation. Die Erkenntnisse der Arbeit liefern eine Grundlage für weitere Projekte, wie etwa die Weiterentwicklung der Applikation.

Abstract (english)

The thesis, "AutoMeasure FleX – GUI Application for Configuring a Measurement Setup via CAN Bus," describes the development of a graphical user interface for controlling a measurement setup and communicating via the CAN bus. The goal of the work was to create a functional, user-friendly, and efficient application that allows for flexible configuration. The choice of programming language and framework—Python 3.13 and Tkinter—is discussed at the beginning and justified by the fact that they ensure rapid prototyping and easy maintenance.

The implementation of CAN bus communication was based on the PCAN-USB API, with particular attention paid to stable and reliable data transmission. Programming principles such as PEP 8, modularization, and error management were implemented to improve software quality. Furthermore, the application was delivered as an executable file to enable trouble-free use without the need for Python installation.

Despite its successful implementation, some aspects remain open for future development, particularly data visualization and the optimization of CAN bus communication. The findings of this work provide a basis for further projects, such as the further development of the application.

Vorwort

In meiner Kindheit in den ersten Jahren des neuen Jahrtausends spielte ich ein Real-Time-Strategy-Spiel am Computer, in dem man als Zivilisation alle historischen Epochen der Menschheitsgeschichte bis sogar in die Zukunft durchschreiten konnte. Besonders spannend waren für mich dabei die Zukunftsepochen – würden sie doch einem naiven jungen Menschen einen flüchtigen Einblick in das, was wohl noch kommen sollte, geben. Die erste in der Zukunft befindliche Epoche wurde dort als das „digitale Zeitalter“ bezeichnet. Ein Schlagwort mit dem mein kindliches Ich noch nicht viel anfangen konnte. Heute blicke ich mit der Gewissheit zurück, dass das digitale Zeitalter mit seiner ungebremsen, schier grenzenlosen technischen Entwicklung längst eingeläutet worden ist. Es erfüllt mich mit dem ambivalenten Gefühl eines wohligen Grusels, ein Teil ebenjener dystopischen Zukunft zu sein, für die dieser 10-jährige Bub so eine Faszination empfunden hat.

Auch wenn der Weg zu diesem Projekt hin ein Produkt der Umstände war und die damit verbundenen *memoriae asperorum* zahlreich sind, krönt es dennoch den Abschluss meiner dreijährigen Ausbildung an der BULME Graz-Gösting – eine Zeit, in der ich trotz aller Mühen und Umwege interessante Einsichten gewinnen und praktische Kompetenzen erwerben konnte.

An dieser Stelle gilt es, allen danken, die zum Entstehen dieser Arbeit beigetragen oder mich in sonstiger Art und Weise dabei unterstützt haben. Namentliche Erwähnung gebührt DI Stefan Kummer, der nicht nur meine Betreuung während der Projektarbeit übernommen hat und mir dabei freie Hand zur Entfaltung ließ, sondern der auch ein beherzter Lehrer und Klassenvorstand ist, von dem ich sehr Vieles lernen konnte – weit über die Grenzen der Welt der Elektronik hinaus.

Inhaltsverzeichnis

Eidesstattliche Erklärung	2
Abstract (deutsch)	3
Abstract (english)	4
Vorwort	5
1 Einleitung	7
2 Einführung in das Controller Area Network (CAN)	8
2.1 Architektur, Topologie, Verkabelung	9
2.2 PCANBasic: API zur Anbindung an den CAN-Bus	11
3 Datenfluss und erzeugte Datenmenge des Messsystems	12
3.1 Grundsätzliche Überlegungen	12
3.2 Kommunikation über SPI	14
3.3 STM32G491RE Datenpuffer im SRAM	14
3.4 Übertragung über den CAN-Bus	15
4 Graphic User Interface (GUI)	18
4.1 Wahl der Programmiersprache: Python 3.13	18
4.2 Theorie und Praxis der GUI-Entwicklung.....	19
4.3 Framework TKinter	27
4.4 Softwarearchitektur & Softwareoptimierung.....	43
4.5 Kommunikation, Verarbeitung und Ausgabe der Messdaten	47
4.6 Finalisierung und letzter Schliff	50
5 Dokumentation der praktischen Umsetzung: GUI-Projekt	58
5.1 „Von der Idee zum Konzept“: Planungs- und Konzeptionsphase	58
5.2 Umsetzungsphase: Implementierung und Herausforderungen.....	63
5.3 Finalisierungsphase – letzter Feinschliff und Abschluss.....	73
6 Zusammenfassung	77
7 Literatur- und Quellenverzeichnis.....	80
8 Abbildungs-, Tabellen- und Abkürzungsverzeichnis	83
8.1 Abbildungsverzeichnis.....	83
8.2 Tabellenverzeichnis	83
8.3 Abkürzungsverzeichnis.....	84

1 Einleitung

Wir leben in einer Welt, in der die automatisierte Datenerfassung und -verarbeitung viele Bereiche unseres Lebens wesentlich beeinflusst. Insbesondere industrielle Anwendungen und Produktionsstätten sind auf eine Vielzahl von Sensoren, die kontinuierlich Messwerte liefern, und auf eine stabile Kommunikation angewiesen. Hier, im Speziellen in der Automobilindustrie und in der Automatisierungstechnik, hat sich das Controller Area Network als Standard für die serielle Datenübertragung etabliert.

Die der vorliegenden Arbeit zugrundeliegende Überlegung bzw. Herausforderung besteht darin, eine benutzerfreundliche Schnittstelle zu entwickeln, die sowohl eine Konfiguration individuell erstellter Messaufbauten über den CAN-Bus als auch eine Übertragung und Speicherung von auf diese Art gewonnenen Messdaten ermöglicht.

Konkret war es das Ziel des vorliegenden Projekts, eine grafische Benutzeroberfläche zur Steuerung und Konfiguration eines Messaufbaus sowie ein Kommunikationsprotokoll für die Kommunikation über den Bus zu entwickeln. Da weder im Bereich der GUI-Entwicklung noch im Bereich der CAN-Bus-Kommunikation auf ein nennenswertes Vorwissen zurückgegriffen werden konnte, sind einige Schritte zu setzen, um die notwendigen Fähigkeiten zu erwerben, die für die Erarbeitung einer flexiblen und intuitive Softwarelösung nötig sind.

Im ersten Schritt gilt es, das notwendige Wissen über die CAN-Bus-Kommunikation zu erwerben und eine passende Programmierschnittstelle für die Anbindung an den CAN-Bus auszuwählen. In weiterer Folge war es geboten, sich eingehend mit der Theorie und Praxis der GUI-Entwicklung zu befassen sowie eine Vorgangsweise zur strukturierten Entwicklung zu erarbeiten. Auch muss im Rahmen der Projektarbeit die passende Programmiersprache für die Umsetzung des Projektvorhabens auserkoren werden. Diese sollte über eine klare Syntax und für das Projekt passende Bibliotheksunterstützung sowie ein kompatibles GUI-Framework verfügen.

So sollte in iterativen Schritten eine Anwendung erstellt werden, die den Zielen des Projekts entspricht. Die Anwendung sollte die manuelle Konfiguration eines aus ADCs, Komparatoren, GPIO-Pins und von MOSFETs getriebenen Relais bestehenden Messaufbaus ermöglichen, aber auch Konfigurationen und über den Bus empfangene Messergebnisse in einer Datei speichern können.

Die vorliegende Arbeit ist in mehrere Kapitel unterteilt, die soeben aufgezählten Überlegungen abbilden: Nach einer grundlegenden Einführung in die Funktionsprinzipien des CAN-Busses und der Dokumentation der Rechercheergebnisse und theoretischen Überlegungen zur Entwicklung von GUI-Applikationen folgt eine Beschreibung der Softwarearchitektur. Beginnend mit der Diskussion über die Auswahl der Programmiersprache und des GUI-Frameworks soll besonders auf die Funktionen des zweitgenannten näher eingegangen werden.

In weiterer Folge soll die praktische Umsetzung der theoretischen Erkenntnisse dokumentiert werden und abschließend werden die Ergebnisse zusammengefasst und kritisch reflektiert.

Das im Rahmen dieses Projekts erarbeitete Softwareprototyp bietet eine solide Grundlage für weiterführende Projekte mit zusätzlichen Funktionen oder komplexere Messaufbauten.

2 Einführung in das Controller Area Network (CAN)

Das Controller Area Network (CAN) ist ein serielles Bussystem. Es wurde von der Robert Bosch GmbH entwickelt und sollte die Verkabelung von Kraftfahrzeugen vereinfachen. Darüber hinaus hat sich der CAN-Bus jedoch auch in der industriellen Automatisierung als zuverlässiges Kommunikationsmedium etabliert (Corrigan, 2016, S. 1). Das Konzept des Controller Area Networks basiert auf dem OSI-Referenzmodell und nutzt in erster Linie die unteren beiden Schichten zur Datenübertragung (Corrigan, 2016, S. 2).

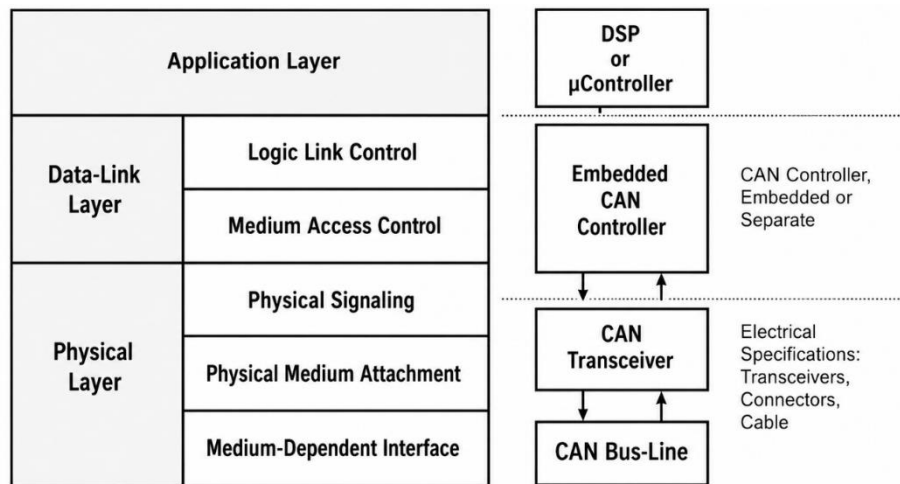


Abbildung 1 ISO 11898 Standard Architecture, eigene Abbildung nach (Corrigan, 2016, S. 2)

2.1 Architektur, Topologie, Verkabelung

Der CAN-Bus gehört zu den lokalen Rechnernetzen (LAN) und wird in hohen Stückzahlen produziert. Dadurch ist CAN zu dem am weitesten verbreiteten Bussystem geworden (Engels, 2000, S. 13). Der CAN-Bus stellt eine Weiterentwicklung bestehender Netzwerkstrukturen dar und ermöglicht eine einfache Zusammenschaltung der Kommunikationsteilnehmer (Engels, 2000, S. 17). Er basiert auf einer linearen Bustopologie mit folgenden Merkmalen:

- Die Signale breiten sich in beide Richtungen aus. Leitungsenden müssen daher mit einem Wellenwiderstand abgeschlossen sein.
- Es ist immer nur ein Netzknoten als Sender aktiv. Die sogenannte Busarbitration regelt den Zugriff, um Kollisionen zu vermeiden.
- Nachrichten werden von allen Teilnehmern empfangen. Diese entscheiden in weiterer Folge, ob die Information für sie relevant und somit zu verarbeiten ist.
- Eine Routing-Funktion ist nicht erforderlich, denn jede Nachricht ist im gesamten Netzwerk verfügbar.
- Neue Teilnehmer können einfach durch Anhängen an den Bus eingefügt werden.
- Die Anzahl der Botschaften und deren Zykluszeiten bestimmen die Busauslastung und damit die maximale Teilnehmerzahl.

- Die Signalgeschwindigkeit begrenzt die maximale Buslänge und die Teilnehmeranzahl.
- Der CAN-Bus ist ein Multimaster-System, bei dem jeder Knoten sowohl als Master als auch als Slave agieren kann (Engels, 2000, S. 17-18)

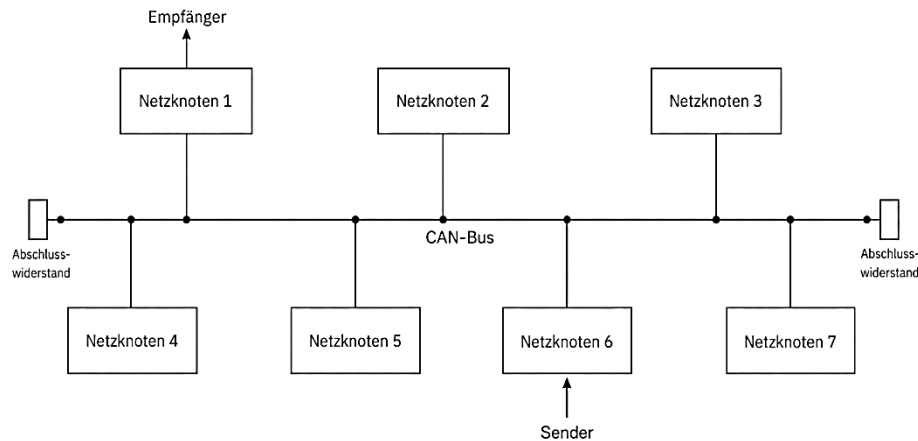


Abbildung 2 Bus-Topologie mit 7 Netzwerkknoten und Abschlusswiderständen, eigene Abbildung nach (Engels, 2000, S. 17)

Im CAN-Netzwerk haben sich zwei unterschiedliche Kommunikationsprinzipien durchgesetzt: das **Multimaster-** und das **Master-Slave-Prinzip**. Der Master initiiert und steuert als aktiver Kommunikationspartner den Datentransfer über den Bus. Er sendet Daten an Slaves oder fordert diese von ihnen an (Engels, 2000, S. 19-20). Das Multimaster-Prinzip hingegen erlaubt es, dass mehrere Teilnehmer als Master agieren. Konflikte, wenn beispielsweise zwei oder mehr Master gleichzeitig senden wollen, werden beim CAN-Bus durch eine „bitweise, nicht-destruktive Arbitration“ gelöst (Corrigan, 2016, S. 5).

Die physikalische Schicht wird beim CAN-Bus über verschiedene **Kabeltypen** umgesetzt:

- **Verdrillte Zweidrahtleitungen** (Twisted Pair) minimieren elektromagnetische Störungen. Die Störfestigkeit kann über zusätzliche Abschirmung weiter erhöht werden (Engels, 2000, S. 21).
- **Koaxialkabel** bieten aufgrund ihrer Abschirmung bereits eine hohe Resistenz gegen Störungen. Sie kommen wegen ihrer höheren Kosten jedoch nur in speziellen Anwendungen zum Einsatz (Engels, 2000, S. 21).

Feldbusse sind serielle, digitale Kommunikationssysteme. Sie wurden speziell für industrielle Anwendungen entwickelt und nutzen üblicherweise digitale Kommunikationsprotokolle anstelle analoger 4–20-mA-Signale. Durch die bidirektionale Kommunikation können zusätzliche Parameter wie Wartungsinformationen oder Diagnosedaten übertragen werden (Engels, 2000, S. 26–29). Der Einsatz von Feldbussen reduziert Kosten, Verkabelungsaufwand und Platzbedarf, erhöht gleichzeitig die Flexibilität und Wartungsfreundlichkeit des Kommunikationssystems (Corrigan, 2016, S. 7).

Der CAN-Bus zeichnet sich durch seine robuste, fehlerresistente Kommunikation sowie eine einfache Erweiterbarkeit und hohe Effizienz aus. Somit ist anzunehmen, dass das Controller Area Network auch in Zukunft eine wichtige Rolle in industriellen Anwendungen spielen wird.

2.2 PCANBasic: API zur Anbindung an den CAN-Bus

Zum Datenaustausch zwischen der GUI-Anwendung und dem CAN-Bus wurde die PCAN-USB API ausgewählt. PCAN-Basic, von der PEAK-System Technik GmbH in Darmstadt entwickelt, stellt als Programmierschnittstelle grundlegende Funktionen zum Senden und Empfangen von CAN-Nachrichten bereit (PCAN-USB Benutzerhandbuch, 2024, S. 29).

Die API ist mit Windows- sowie mit Linux-Systemen kompatibel und unterstützt die CAN-Spezifikationen 2.0A, 2.0B sowie CAN-FD. Sie erlaubt den Zugriff auf bis zu 16 Kanäle pro Gerät und ermöglicht das gleichzeitige Betreiben mehrerer Applikationen auf demselben physikalischen Kanal (PCAN-USB Benutzerhandbuch, 2024, S. 30). Die technische Umsetzung und Herangehensweise im Zuge des Projekts werden in Kapitel 4.5.1 näher erläutert.

3 Datenfluss und erzeugte Datenmenge des Messsystems

Im vorliegenden System werden sowohl die Messergebnisse der auf den Messaufbauten angebrachten Analog-Digital-Wandler (ADCs) als auch digitale Komparator-Ausgänge ausgelesen und über den CAN-Bus an einen Computer zur weiteren Verarbeitung übermittelt. Daher ist es unbedingt erforderlich, die von diesem System „verursachten“ Datenmengen abschätzen zu können. Zu diesem Zweck soll eine exemplarische Berechnung der erzeugten Datenrate Thema dieses Kapitels sein.

Als Grundlage für die theoretischen Überlegungen werden der STM32G491RE-Mikrocontrollerchip sowie der ADC MCP3464 angenommen.

3.1 Grundsätzliche Überlegungen

Jeder der bis zu vier verbauten ADCs besitzt drei analoge Eingangskanäle und ermöglicht eine Auflösung von 12 Bit oder 16 Bit pro Sample (Microchip Technology Inc., 2024). Die pro Sekunde erzeugte Datenmenge pro Kanal hängt somit von der gewählten Abtastrate f_s (in Hz) und der Auflösung b (in Bit) ab. Die Datenrate pro Kanal wird mit folgender Formel berechnet:

$$R_{Kanal} = f_s \times b \text{ [Bit/s]}$$

Wird eine Abtastrate $f_s = 1000 \text{ Hz}$ und eine Auflösung von $b = 16 \text{ Bit}$ gewählt, ergibt sich für einen einzelnen Kanal eine Datenrate von:

$$R_{Kanal} = 1 \text{ kHz} \times 16 \text{ Bit} = \frac{1000 \times 16 \text{ Bit}}{s} = 16\,000 \text{ Bit/s} = 2 \text{ kB/s}$$

Bei einer Reduktion der Auflösung auf 12 Bit verringert sich die Datenrate entsprechend auf 12 000 Bit/s bzw. 1,5 kB/s.

Da jeder ADC drei Kanäle besitzt, lässt sich die Datenrate pro ADC folgendermaßen berechnen:

$$R_{ADC} = 3 \times f_s \times b \text{ Bit/s}$$

Nimmt man als Abtastrate f_s 1000 Hz sowie als Auflösung 16 Bit an, so ergibt sich daraus:

$$R_{ADC} = 3 \times 1 \text{ kHz} \times 16 \text{ Bit} = 48\,000 \text{ Bit/s} = 6 \text{ kB/s}$$

Für den gesamten Aufbau mit vier ADCs und damit zwölf Kanälen beträgt die erzeugte Datenrate das Vierfache: 24 kB/s.

Die Datenrate steht dabei in einem Proportionalverhältnis zur gewählten f_s . Würde man die Abtastrate beispielsweise auf 10 kHz erhöhen, so ergibt sich folgende Datenmenge:

$$4 \text{ ADCs} \times 3 \text{ Kanäle} \times 10 \text{ kHz} \times 16 \text{ Bit} = 1\,920\,000 \text{ Bit/s} = 240 \text{ kB/s}$$

Daneben verarbeitet der Mikrocontroller noch zwölf digitale Komparatorausgänge. Jeder Komparator liefert dabei lediglich ein 1-Bit-Signal, wodurch sich die Datenrate wie folgt errechnet:

$$R_{Comp} = f_{Comp} \times 1 \text{ Bit}$$

Bei einer Abtastrate von $f_{Comp} = 1000 \text{ Hz}$ ergibt sich folglich für alle zwölf Komparatoren:

$$12 \times 1 \text{ kHz} \times 1 \text{ Bit} = 12\,000 \text{ Bit/s} = 1,5 \text{ kB/s}$$

Zusammenfassend lässt sich festhalten:

Datenquelle	Auflösung pro Sample	Datenrate bei $f_s = 1 \text{ kHz}$	Datenrate bei $f_s = 2 \text{ kHz}$
12 Kanäle (4 ADCs)	$12 \times 16 \text{ Bit} = 192 \text{ Bit}$	192 kBit/s (24 kB/s)	384 kBit/s (48 kB/s)
12 Komparatoren	$12 \times 1 \text{ Bit} = 12 \text{ Bit}$	12 kBit/s (1,5 kB/s)	24 kbit/s (3 kB/s)
Gesamt	$12 \times (16 \text{ Bit} + 1 \text{ Bit}) = 204 \text{ Bit}$	204 kBit/s (25,5 kB/s)	408 kBit/s (51 kB/s)

Tabelle 1 Datenraten des Messaufbaus bei unterschiedlichen Abtastraten

Daraus wird gleich ersichtlich, dass die ADC-Daten den weitaus größten Teil der erzeugten Datenmenge ausmachen, während die Komparatoren im Vergleich nur einen geringen Teil der gesamten Datenrate erzeugen.

3.2 Kommunikation über SPI

Die Verbindung zwischen dem Mikrocontroller und den ADCs erfolgt über die SPI-Schnittstelle (Serial Peripheral Interface), die als synchrones, serielles Voll-Duplex-Protokoll arbeitet (VectorNav Technologies, 2024). Der Mikrocontroller STM32G491RE agiert dabei als Master und steuert die vier externen ADCs als Slaves. Jeder ADC verfügt über eine eigene Chip-Select-Leitung (CS), sodass der Mikrocontroller die einzelnen Wandler nacheinander selektiv abfragen kann.

Die eingesetzten MCP3464-ADCs unterstützen laut Datenblatt SPI-Taktfrequenzen von bis zu 20 MHz (Microchip Technology Inc., 2024). Die Übertragung eines einzelnen 16-Bit-Messwerts dauert bei maximaler Taktrate somit theoretisch:

$$\frac{16 \text{ Bit}}{20\,000\,000 \text{ Bit/s}} = 0,8 \mu\text{s}$$

Daraus ergibt sich in weiterer Folge, dass die Übertragung aller drei Kanäle eines ADCs rund $2,4 \mu\text{s}$ in Anspruch nimmt, für das Auslesen aller vier ADCs mit insgesamt zwölf Kanälen werden $9,6 \mu\text{s}$ benötigt.

Bei einer angenommenen Frequenz f_s von 1 kHz beträgt die Abtastperiode 1 ms, selbst bei einer Abtastrate von 10 kHz (entspricht $0,1 \text{ ms}$ Periodendauer) verbleiben ausreichend Reserven für die Übertragung via SPI.

Die Synchronisation der Messwertabfrage erfolgt durch Timer-Interrupts oder alternativ über die Datenbereit-Signale (DRDY) der ADCs, die ein Interrupt-Signal liefern, sobald neue Messdaten bereitstehen. Dadurch wird eine jitterarme und präzise Abtastung gewährleistet.

3.3 STM32G491RE Datenpuffer im SRAM

Um die Messwerte in weiterer Folge über den CAN-Bus zu übertragen, werden die Daten im internen SRAM des Mikrocontrollers gepuffert. Der STM32G491RE verfügt über 112 kB RAM (STMicroelectronics, 2024), wovon ein erheblicher Teil für einen sogenannten Ringpuffer (Circular Buffer) genutzt werden kann – in weiterer Folge wird angenommen, dass von den 112 kB etwa 24 kB von der Firmware, weitere 8 kB

vom Stack und nochmals 4 kB für den Heap verwendet werden, somit verbleiben nach Abzug einer Reserve ca. 70 kB für den Ringpuffer.

In diesem Puffer werden die ankommenden Daten kontinuierlich abgelegt. Ist das Ende des Puffers erreicht, beginnt der Schreibvorgang wieder am Anfang, wobei die ältesten Daten überschrieben werden, falls diese nicht rechtzeitig ausgelesen werden (Wikipedia, Ringpuffer).

Pro Abtastzyklus fallen etwa 26 Byte an Daten an (24 Byte für die zwölf ADC-Kanäle, 2 Byte für die zwölf Komparatoren). Daraus ergibt sich pro Sekunde bei 1 kHz Abtastrate:

Mit einem angenommenen Speicherreservoir des Ringpuffers von 70 kB

$$\frac{70 \text{ kB}}{26 \text{ Byte}} \approx 2\,692$$

Somit können bis zu 2692 Abtastzyklen durchgeführt und im Ringpuffer gespeichert werden, bevor alte Messdaten verloren gehen. Wird der Ringpuffer DMA-gestützt (Direct Memory Access) befüllt, steigert dies die Effizienz noch weiter, denn der DMA-Controller überträgt die Daten direkt von der SPI-Peripherie in den Speicher, und umgeht dabei die CPU. Dadurch wird die Interrupt-Last erheblich reduziert, was in erster Linie bei hohen Abtastraten positiv ins Gewicht fällt (Electronics Stack Exchange, 2023).

3.4 Übertragung über den CAN-Bus

Der weitere Weg der im SRAM gepufferten Daten führt über den CAN-Bus zum PC des Endnutzers. Der verwendete STM-Mikrocontroller besitzt hierfür einen integrierten FDCAN-Controller, der mittels eines externen Transceivers an den CAN-Bus (ISO 11898-2, High-Speed CAN) angebunden ist (STMicroelectronics, 2024). Die maximale Baudrate im verwendeten CAN-Standard beträgt 1 Mbit/s, die minimale 125 kBit/s.

Dabei ist zu bedenken, dass sich ein CAN-Datenframe aus mehreren Feldern zusammensetzt: Zusätzlich zu den eigentlichen Nutzdaten enthält ein Frame einen Identifier (Arbitration Field), Steuerinformationen (Control Field), Prüfsummen (CRC Field) sowie Synchronisations- und Abschlussbits (Wikipedia, CAN-Bus, kein Datum).

Ein vollständiger CAN-Frame mit maximaler Nutzlast (8 Byte Data Field) umfasst dadurch etwa 111 Bit, zuzüglich mindestens 3 Bit Interframe Space. Die resultierende Datenmenge pro Nachricht beträgt damit ca. 114 Bit:

Die minimal übertragbare Datenmenge pro Sekunde beträgt somit:

$$\frac{125\,000\text{ Bit/s}}{114\text{ Bit}} \approx 1\,096\text{ Frames/s}$$

$$1\,096 \times 8\text{ Byte} = 8\,768\text{ Byte/s} \approx 70,14\text{ kbit/s}$$

Die maximal übertragbare Nutzdatenmenge pro Sekunde hingegen beläuft sich auf etwa:

$$\frac{1\,000\,000\text{ Bit/s}}{114\text{ Bit}} \approx 8\,770\text{ Frames/s}$$

$$8\,770 \times 8\text{ Byte} = 70\,160\text{ Byte/s} \approx 0,56\text{ Mbit/s}$$

Dies entspricht einer Netto-Datenrate von etwa 56 % der physikalischen Bitrate. Der restliche Anteil entfällt auf den Overhead des CAN-Protokolls.

Vergleicht man die errechnete maximale Nutzdatenrate mit den vom Messaufbau theoretisch erzeugten Datenmengen, dann ergibt sich bei einer Abtastrate von 1 kHz folgende Busauslastung:

$$\frac{25,5\text{ kB/s}}{70,16\text{ kB/s}} \approx 36\%$$

Wird die f_s auf 2 kHz erhöht, so verdoppelt sich auch die Busauslastung auf ca. 72 %. Es empfiehlt sich allerdings in der Praxis nicht, ein CAN-Netz dauerhaft über 80 % auszulasten, um auch bei Arbitration oder Spitzenlasten ausreichend Reserve zu haben (Voss, 2017, S. 58).

Da ein einzelner CAN-Frame maximal 8 Byte an Nutzdaten übertragen kann, ist es notwendig, die erzeugten 26 Byte pro Abtastzyklus auf mehrere Frames aufzuteilen:

- Frame 1: 6 Byte Messwerte von ADC 1
- Frame 2: 6 Byte Messwerte von ADC 2
- Frame 3: 6 Byte Messwerte von ADC 3
- Frame 4: 6 Byte Messwerte von ADC 4 + 2 Byte Komparator-Status

Durch diese Aufteilung wird die maximale Nutzlast jedes Frames bestmöglich ausgenutzt, der Overhead pro übertragenem Messzyklus bleibt dadurch minimal.

Zwar unterstützt der STM32G491RE auch den erweiterten CAN-FD-Standard (Flexible Data Rate), der höhere Datenraten sowie eine maximale Framegröße von bis zu 64 Byte ermöglicht. Im Rahmen dieser Arbeit wurde jedoch bewusst auf die Nutzung von CAN-FD verzichtet, da dies nicht Teil der Projektanforderungen war. Für zukünftige Projekte oder Erweiterungen wäre der Einsatz von CAN-FD jedoch eine mögliche Option, um auch bei höheren Abtastraten und größeren Datenmengen pro Zyklus eine effiziente Übertragung zu bewerkstelligen.

Die Berechnungen zeigen, dass bei Verwendung des CAN 2.0-Standards und bei einer Abtastrate von 1 kHz eine Datenmenge von 204 kBit/s bzw. 25,5 kB/s zu erwarten ist. Augenscheinlich wurde allerdings ebenfalls, dass der CAN-Bus – wenn er nicht von anderen Teilnehmern belegt ist – Daten, die bei einer Abtastrate von bis zu 2 kHz entstehen, zuverlässig und mit ausreichend Reserven übertragen kann und dabei mit Datenraten von bis zu 408 kBit/s bzw. 51 kB/s zu rechnen ist.

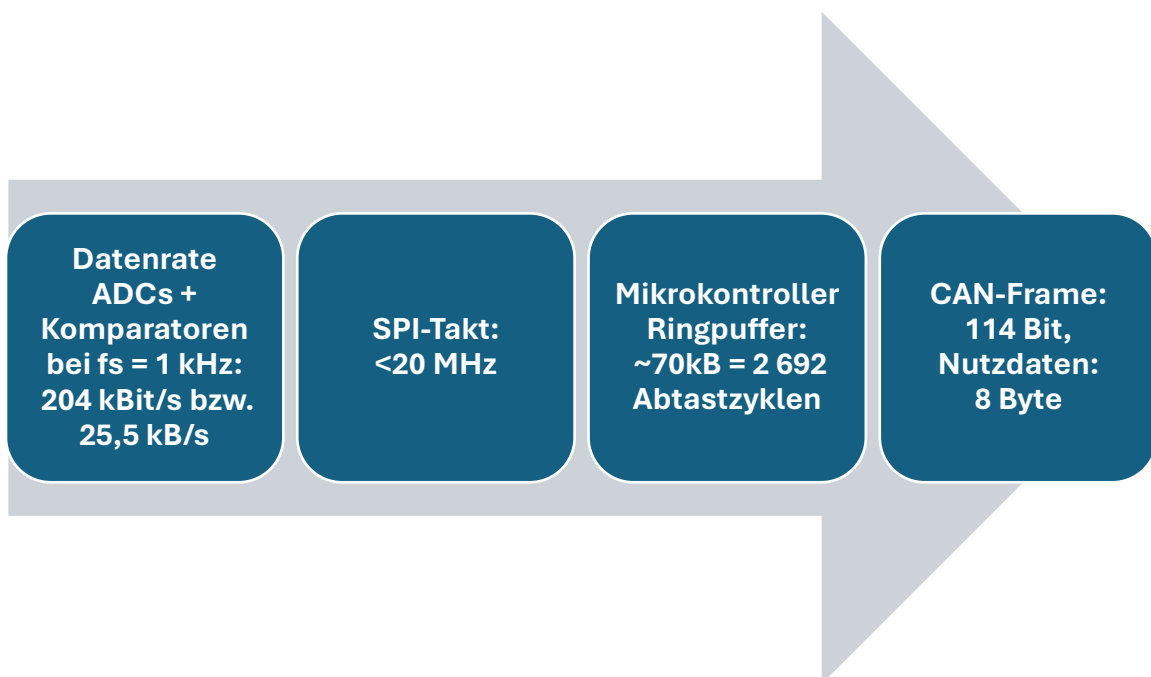


Abbildung 3 Veranschaulichung der Datenübertragung von den Sensoren bis zur PC-Applikation des Endnutzers

4 Graphic User Interface (GUI)

Im Rahmen der vorliegenden Arbeit werden grundlegende Prinzipien der Programmierung, wie beispielsweise Verzweigungen, Schleifen und Datentypen, nicht näher erläutert. Die Kenntnis dieser wird bei der Leserschaft vorausgesetzt. Gleichwohl werden tiefergehende, gleichermaßen für die gewählte Programmiersprache wie auch für das Projekt herausgearbeitete Spezifika, wie beispielsweise Charakteristika der Programmiersprache, Besonderheiten des GUI-Frameworks und fortgeschrittene Konzepte der Programmierung, näher beleuchtet. Zunächst soll jedoch auf die Wahl der Programmiersprache eingegangen werden.

4.1 Wahl der Programmiersprache: Python 3.13

Als der wohl zentralste Aspekt bei der Konzeption und Entwicklung einer Anwendung gilt die Wahl der Programmiersprache. Im Rahmen des Diplomarbeitsprojekts fiel die Wahl auf Python in der Version 3.13. Die Gründe für diese Entscheidung werden im Folgenden detailliert dargelegt:

Am 07. Oktober 2024 erschien mit der Version 3.13 der aktuellste Interpreter für Python. Ein vollumfänglicher Support ist bis Oktober 2026, Sicherheitskorrekturen sind gar bis Oktober 2029 gewährleistet (Wouters, 2023). Die Aktualität sowie der langfristige Support bilden ein stabiles Fundament für ein Projekt, das über mehrere Monate hinweg entwickelt wird.

Python hat sich seit seinem erstmaligen Erscheinen im Jahr 1991 zu einer der beliebtesten Programmiersprachen entwickelt. Die Sprache zeichnet sich nämlich durch eine einfache Syntax aus, die auf viele Sonderzeichen (wie geschweifte Klammern und Semikolons) verzichtet und den Code stattdessen durch Einrückungen strukturiert (Theis, 2024, S. 19). Diese Syntax führt zu übersichtlicherem und besser wartbarem Code, was für die Entwicklung der GUI-Applikation im Rahmen dieses Projekts von enormem Vorteil ist (Steyer, 2018, S. 3).

Python ist eine **interpretierte Sprache**. Der Code muss somit nicht vorab kompiliert werden, sondern wird direkt vom Interpreter ausgeführt. Im Gegensatz zu Compilersprachen wie C oder C++ muss der Quellcode also nicht vollständig in Maschinensprache übersetzt werden, bevor er ausgeführt wird (Fehr, 2019, S. 38-39).

Dadurch können Änderungen schneller getestet werden, was die Fehlersuche erleichtert und die Entwicklungszeiten deutlich verkürzt (Weigend, 2019, S. 21). Zwar sind interpretierte Programme tendenziell weniger performant als kompilierte (Steyer, 2018, S. 42), im Rahmen des vorliegenden Projekts liegt der Schwerpunkt jedoch auf der schnelleren sowie einfacheren Prototypentwicklung.

Ein weiterer Vorteil von Python ist die **dynamische Typisierung**. Der Datentyp von Variablen muss nicht vorab deklariert werden und kann sich zur Laufzeit flexibel ändern. Typische Fehlerquellen statisch typisierter Sprachen werden auf diese Weise reduziert (Bonacina, 2018, S. 48-49). Die **automatische Speicherbereinigung** von Python ermöglicht überdies eine effizientere Entwicklung (Knoll & Heedt, 2022, S. 45).

Python enthält außerdem eine umfangreiche **Standardbibliothek** mit zahlreichen Modulen für spezielle Anwendungen, was stellenweise als „batteries-included“-Philosophie bezeichnet wird (Jones, 2024, S. 7). Python ermöglicht es als plattformunabhängige Programmiersprache, Anwendungen für Windows, Linux oder macOS zu entwickeln (Knoll & Heedt, 2022, S. 45).

Für die **Entwicklung einer GUI-Anwendung** bietet Python ebenfalls große Vorteile. Die in Python integrierte Bibliothek Tkinter beispielsweise bietet eine Reihe grundlegender Funktionen für die Erstellung grafischer Benutzeroberflächen (Jones, 2024, S. 7).

Nicht zuletzt ist zu erwähnen, dass Python sowohl über eine große Community verfügt, als auch in zahlreichen Industrie- und Forschungsbetrieben angewendet wird (Schäfer, 2019, S. 1).

4.2 Theorie und Praxis der GUI-Entwicklung

In diesem Kapitel werden **Frameworks** für die GUI-Entwicklung einander gegenübergestellt und die Entscheidung für **Tkinter** begründet, ebenso wird auf die Prinzipien einer gelungenen GUI näher eingegangen, wie auch auf die konkreten Schritte, die im Vorfeld und während der Entwicklung einer solchen Anwendung idealerweise zu setzen sind.

4.2.1 Überblick über GUI-Frameworks unter Python

Alle modernen GUI-Frameworks ähneln sich in einem Punkt: Sie stellen grundlegende **Komponenten** wie Buttons, Checkboxes, Eingabefelder etc. bereit, die in Container-Komponenten („Frames“) verschachtelt sind. Ein **Layout-Manager** übernimmt dabei oftmals die Anordnung der einzelnen Komponenten. Diese Struktur bildet das Fundament jeder grafischen Benutzeroberfläche in Python (Steyer, 2018, S. 233-235).

Python ist mit mehreren GUI-Toolkits kompatibel. Sie unterscheiden sich in ihren Konzepten, Anpassungs- und Einsatzmöglichkeiten. Nachfolgend eine überblicksmäßige Gegenüberstellung:

Tkinter	In der Standardbibliothek von Python enthalten und ermöglicht eine sofortige Nutzung ohne zusätzliche Installation
PyQt	Bietet umfangreichere Funktionalitäten, ebenso ein modernes Erscheinungsbild. Zusätzliche Installationen sind dabei jedoch notwendig. Außerdem muss der Code aufgrund von Lizenzvorgaben für Endnutzer veränderbar sein
PySide6	Umfangreiches Qt-Framework mit moderner UI; zusätzliche Installation; bei Distribution sind die jeweiligen LGPL/GPL- bzw. kommerziellen Lizenzbedingungen zu beachten.
wxPython	Wrapper des GUI-Toolkits wxWidgets für Python. Stellt robuste, plattformübergreifende GUI-Komponenten bereit. Die Arbeit mit wxPython ist jedoch mit einer steileren Lernkurve verbunden.
PyGObject	API für das Framework GObject, bietet ebenfalls ein plattformunabhängiger Toolkit mit höheren Ansprüchen

Tabelle 2 Übersicht: GUI-Frameworks (Ernesti & Kaiser, 2023, S. 1228-1231)

Die Entscheidung für ein Framework zur Umsetzung des Projekts fiel auf Tkinter, was wie folgt begründet wird.

Tkinter basiert auf dem Tk GUI-Toolkit, welches ursprünglich für die Skriptsprache Tcl entwickelt wurde. Tkinter ist seit langem das Standardframework für die GUI-Entwicklung in Python. Dieses Toolkit bietet eine Vielzahl von grundlegenden Widgets

und zeichnet sich durch einfache Hierarchien aus, bei denen ein oder mehr Frames als Container für weitere Widgets in einem Hauptfenster platziert werden. Tkinter erlaubt eine objektorientierte Herangehensweise, was sowohl die Wartbarkeit als auch die modulare Erweiterung des Codes erleichtert (Steyer, 2018, S. 233-235).

Neben der leichten Verfügbarkeit als Python-Standardbibliothek und einer einfachen Handhabung der GUI-Elemente spricht auch die Stabilität dieser Bibliothek für sich. Denn als reines GUI-Toolkit ohne Threading-, Netzwerk- oder Dateisystem-APIs und mit seiner einfachen Syntax ist Tkinter die ideale Wahl für häufige Iterationen und rasches Prototyping – kurz: für Projekte, in denen die Entwicklungszeit eine Rolle spielt (Moore A. D., 2021, S. 31-32). Diesem Vorteil kommt beim vorliegenden Projekt unter anderem auch deshalb besondere Bedeutung zu, um die Funktionalität der Kommunikation der Applikation über den CAN-Bus regelmäßig und rasch zu überprüfen und zu gewährleisten.

Tkinter ist ebenso wie Python plattformunabhängig und somit ohne größeren Aufwand auf unterschiedlichen Betriebssystemen betriebsfähig (Jones, 2024, S. 7-8). Überdies bleibt der Wartungsaufwand des Codes aufgrund der Abwärtskompatibilität über viele Jahre hinweg gering (Moore A. D., 2021, S. 31-32)

Natürlich darf an dieser Stelle nicht unerwähnt bleiben, dass Tkinter auch Nachteile aufweist. So entsprechen nicht alle Widgets dem „Look-and-Feel“ aktueller Designs von grafischen Oberflächen, außerdem fehlen manche komplexere Widgets, wie zum Beispiel ein integrierter Rich-Text-Editor (Jones, 2024, S. 31-32).

Zusammenfassend ist Tkinter als GUI-Framework die geeignetste Wahl für das vorliegende Projekt, da Funktionalität, Stabilität und rasches Prototyping im Vordergrund stehen. Außerdem bietet Tkinter trotz gewisser Einschränkungen alle notwendigen Funktionen, um eine den Anforderungen des Projekts entsprechende Oberfläche zu entwickeln.

4.2.2 Prinzipien und Best Practices einer gelungenen GUI

Es ist offenkundig, dass eine gute GUI-Anwendung Funktionalität, Benutzerfreundlichkeit und ästhetisches Design in sich vereint. Dieses Kapitel zeigt jene Aspekte auf, die in der Fachliteratur für die Erstellung einer gelungenen Benutzeroberfläche Erwähnung finden.

Als **Design Patterns** werden erprobte Lösungsansätze für wiederkehrende Probleme in der Softwareentwicklung bezeichnet. Im Bereich der GUI-Entwicklung sind folgende Design Patterns geläufig:

Pattern	Beschreibung
Model-View-Controller (MVC)	Eine Trennung der Daten (Model), der Darstellung (View) und der Steuerlogik (Controller) erhöht die Übersichtlichkeit und Wartbarkeit der Anwendung.
Observer/ Observable	Änderungen von Daten sollen an alle betroffenen GUI-Komponenten kommuniziert werden, um sicherzustellen, dass sich beispielsweise Aktualisierungen von Variablen in der gesamten Anwendung niederschlagen.
Command	Benutzeraktionen, z.B. Klicks oder Tastatureingaben, werden durch modulare Codeblöcke (Funktionen, Objekte) isoliert und sollen bei Bedarf während der Laufzeit rückgängig gemacht werden können.
Factory und Singleton	Bestimmte Ressourcen (z. B. das Hauptfenster) dürfen nur als Unikate existieren.
Composite und Strategy	Composite beschreibt das Zusammensetzen kleinerer, einfacher Komponenten zu komplexeren Strukturen, während mit Strategy die Modularisierung der Behandlung von Benutzereingaben gemeint ist. Beide Prinzipien fördern eine flexible und erweiterbare Softwarearchitektur.

Tabelle 3 Design Patterns in der GUI-Entwicklung (Ullas, 2024, S. 103-104)

Über das äußere Erscheinungsbild hinweg ist für eine gelungene GUI-Applikation auch eine solide technische Basis nötig. Als wichtige Prinzipien werden in der Fachliteratur die folgenden genannt:

Modularer und entkoppelter Aufbau	Der Code sollte in kleine, in sich geschlossene Komponenten strukturiert werden, einzelne Teile sollen unabhängig voneinander entwickelt, getestet und gewartet werden können.
Einhaltung der SOLID-Prinzipien	SOLID steht für Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation und Dependency Inversion – kurz gesagt: Durch den Code zieht sich eine klare Struktur.
Dokumentation und Testen	Durch lückenlose Dokumentation sowie umfangreiche Unit-Tests und regelmäßige Code Reviews wird die langfristige Wartbarkeit und Stabilität des Codes gewährleistet.
Kontinuierliche Integration und Refactoring	Automatisierte Build- und Testprozesse halten den Code in einem funktionsfähigen Zustand, regelmäßiges Refactoring verbessert die Codequalität.

Tabelle 4 Technische Richtlinien guter Entwicklungspraxis (Ullas, 2024, S. 105)

Nicht nur, aber insbesondere bei GUI-Anwendungen steht der Anwender im Mittelpunkt – Stichwort User Experience. Wichtige **UX-Prinzipien** sind:

Einfachheit und Konsistenz	Die Benutzeroberfläche sollte klar strukturiert und übersichtlich sein, damit sie vom Endnutzer intuitiv bedient werden kann. Konsistente Layouts, einheitliche Farbgebung und Schriftelemente sollten das Erscheinungsbild prägen.
Klare visuelle Hierarchie	Wichtige Informationen werden durch gezielten Einsatz von Designelementen (Schriftgrößen, Farben, Kontrasten etc.) hervorgehoben, auch die Navigation wird dadurch erleichtert.
Responsive Design	Um sicherzustellen, dass die Applikation auf unterschiedlichen Endgeräten mit diversen Bildschirmgrößen und -auflösungen fehlerfrei läuft, ist eine adaptive Gestaltung sinnvoll

Intuitive Navigation und Feedback	Die Interaktion mit der Anwendung soll selbsterklärend sein. Tooltips, Statusanzeigen oder Fortschrittsbalken geben dem Nutzer ein Feedback zu seinen Aktionen.
Barrierefreiheit	Auch für Menschen mit Behinderungen soll die Anwendung bedienbar sein, beispielsweise indem eine anpassbare Farbkontrastierung und alternative Navigationselemente vorhanden sind.

Tabelle 5 User Experience Prinzipien (Ulias, 2024, S. 112-114) und (Jones, 2024, S. 7-8)

Zusammenfassend ergeben sich aus den in diesem Kapitel erarbeiteten Prinzipien konkrete Anweisungen, die zu einem intuitiven und benutzerfreundlichen User Interface beitragen:

- **„Keep it Simple“:** Die GUI-Anwendung sollte so einfach wie möglich gestaltet sein, denn ein minimalistischer Ansatz verbessert die Benutzbarkeit, die User Experience und reduziert mögliche Fehlerquellen (Jones, 2024, S. 8).
- **Modularisierung:** Die Aufteilung der GUI in voneinander unabhängige Module erleichtert es, Änderungen umzusetzen und den Code zu überarbeiten sowie zu warten (Ulias, 2024, S. 105).
- **Fehlerbehandlung und Feedback:** Um Probleme schnell zu erkennen und zu beheben, sollen Fehlerbehandlungsmechanismen eingebaut werden (Ulias, 2024, S. 105). Ein konkretes Beispiel hierfür wäre etwa die Verifizierung von Nutzereingaben in Entry-Feldern (näher wird darauf in Kapitel 4.3.1.2 eingegangen). Klare Anwendungshinweise für den Benutzer können überdies helfen, Probleme schnell zu identifizieren und zu beheben (Jones, 2024, S. 10).
- **Benutzerzentriertes Design:** Ein umfassendes Verständnis für die Bedürfnisse und Ziele des Endnutzers bei der Verwendung der zu erstellenden Anwendung ist Voraussetzung, um eine intuitive und effiziente Nutzeroberfläche zu gestalten. Hierbei können Usability-Tests und Feedbackrunden zur Optimierung der Anwendung beitragen (Ulias, 2024, S. 112-114).

- **Rückgriff auf bewährte Design Patterns:** Erprobte Konzepte wie MVC oder Observer und Command helfen, eine modulare und erweiterbare Softwarearchitektur zu erschaffen. Die Erfüllung der SOLID-Prinzipien und kontinuierliche Tests gewährleisten eine einfachere Wartbarkeit des Codes.

Diese Ansätze sind nicht nur theoretisch entworfen worden, sondern werden auch als Best Practice von erfahrenen Entwicklern angewendet (Jones, 2024, S. 7-10).

4.2.3 GUI-Erstellung: Konkrete Vorgehensweise

Die Erstellung einer guten GUI erfordert einen strukturierten und iterativen Entwicklungsansatz. Neben theoretischen und konzeptionellen Überlegungen sind beim Entwurf einer GUI-Anwendung ganz handfeste Schritte zu setzen. Vier zentrale Fragestellungen sind im Zuge der **Planungsphase** zu erörtern:

Erstens ist ein Verständnis für die Problemstellung zu gewinnen. Dabei sind die Anforderungen an die zu erstellende Anwendung, Wünsche des Kunden bzw. Endnutzers zu analysieren und zu dokumentieren (Moore A. D., 2021, S. 66-69).

In einem zweiten Schritt ist zu erörtern, welche grafischen Elemente (Widgets) zur Erfüllung der Aufgaben der Anwendung benötigt werden bzw. dienlich sind (Weigend, 2019, S. 432). Für jedes Datenelement wird ein geeigneter Widget-Typ ausgewählt. So eignet sich das Widget Spinbox besonders gut für numerische Werte, Dropdown-Listen oder Radiobuttons hingegen für diskrete Auswahlmöglichkeiten (Moore A. D., 2021, S. 81-85). Im Kapitel 4.3.1.2 wird näher auf die Tkinter-Widgets eingegangen.

Als drittes muss überlegt werden, wie die einzelnen Widgets innerhalb des Programmfensters angeordnet werden, sodass sie auch bei dynamischer Veränderung der Fenstergröße ein sinnvolles und flexibles Layout ermöglichen. Diese Aufgabe wird in der Regel von Geometry- oder Layout-Managern übernommen, diese werden im Abschnitt 4.3.1.1 erläutert.

Viertens: Benutzerereignisse (Klicks, Tastenanschläge, Mausbewegungen etc.) müssen von der Applikation wahrgenommen und darauf reagiert werden. Es muss festgestellt werden, wie die Applikation auf definierte Benutzereingaben reagieren soll (Weigend, 2019, S. 432).

Sind diese Überlegungen erst einmal geklärt, so kann die nächste Entwicklungsphase eingeleitet werden, die **Konzeption und Umsetzung**. Auch diese Phase lässt sich wiederum in vier Teilschritte herunterbrechen:

Erster Ausgangspunkt für das konkrete Design der Benutzeroberfläche ist ein visueller Entwurf der Oberfläche, das kann durchaus mit Papier und Stift erfolgen, aus der die Master-Slave-Architektur der Widgets bereits hervorgeht. Die elementaren Bausteine der Anwendung und ihre Beziehung zueinander werden hier bereits festgelegt, jedoch nicht unabänderlich.

Im zweiten Schritt wird das grafische Konzept mit Widgets nachvollzogen. Callback-Funktionen, die auf Benutzeraktionen reagieren, werden vorerst nur als Platzhalter (unter Python z. B. mit der Anweisung „pass“) erstellt (Weigend, 2019, S. 471).

Zur besseren Organisation der Widgets werden sie mithilfe eines Layout-Managers – bei Tkinter sind dies: `grid()`, `pack()` und `place()` – im Programmfenster angeordnet. Das Programm ist nun bereits lauffähig, das visuelle Erscheinungsbild kann somit auf dem Bildschirm begutachtet und bewertet werden kann (Weigend, 2019, S. 470-470). Eine klare Strukturierung, beispielsweise durch das Zusammenfassen von Feldern in Rahmen (Frames), hilft dabei, die Benutzerführung zu verbessern. Die Reihenfolge und Platzierung der Widgets sollten sich an den Arbeitsabläufen der Nutzer orientieren, um eine intuitive Bedienung zu gewährleisten. (Moore A. D., 2021, S. 85-86).

Sobald das Layout den Anforderungen des Endnutzers entspricht, werden zuvor eingefügte Funktionen mit ausführbarem Code erweitert. Ereignisse wie Button-Klicks oder Eingaben werden somit korrekt bearbeitet (Weigend, 2019, S. 473-476). Diese Phase ist iterativ, sodass das System fortlaufend getestet und anhand von Nutzerfeedback verbessert wird. In enger Abstimmung mit dem Kunden oder Endnutzer wird die Anwendung laufend evaluiert und notwendige Anpassungen vorgenommen (Moore A. D., 2021, S. 82-91).

Wird dieser mehrstufige Prozess, der gleichermaßen die konzeptionelle wie auch die technische Seite berücksichtigt, während der Planung, Konzeption und Entwicklung durchlaufen, ist die Wahrscheinlichkeit hoch, dass am Ende eine Anwendung entsteht,

die funktional und benutzerfreundlich ist. Wird das Feedback von Endnutzern, Kunden oder Kollegen regelmäßig eingeholt und frühzeitig einbezogen, können Irrwege bei der Entwicklung verhindert, Fehler minimiert und die Usability deutlich verbessert werden.

4.3 Framework TKinter

Tkinter stellt eine objektorientierte Schnittstelle zum Tk-GUI-Toolkit dar, das ursprünglich als Erweiterung für Tcl (Tool Command Language) entwickelt wurde. Seinen Ursprung hat Tk in den Unix-Umgebungen, doch aufgrund seiner einfachen Gestaltung fand es schnell in akademischen und technischen Anwendungsgebieten Verbreitung. Bereits seit der Veröffentlichung von Python in der Version 1.1 im Jahr 1994 ist Tkinter ein Bestandteil der Python-Standardbibliothek, was es zum naheliegendsten Framework für die GUI-Entwicklung unter Python macht (Moore A. D., 2021, S. 31).

4.3.1 Tkinter: Grundlegende Funktionen und Konzepte

Grundlegende Funktionen und Aufbau einer Tkinter-Anwendung

Grundlegende GUI-Anwendungen lassen sich mit Tkinter bereits mit wenigen Zeilen Code erstellen, was besonders in der frühen Entwicklungsphase zur schnellen und unkomplizierten Umsetzung von Ideen beiträgt. Je größer das Projekt ist, umso dringlicher ist ein Unterteilen des Codes in logische Module (Paz, 2018, S. 31).

Bei der Entwicklung einer Tkinter-Anwendung wird ein objektorientierter Ansatz verfolgt. Bereits das Hauptfenster wird als Objekt der Klasse Tk instanziiert (Theis, 2024, S. 339). Bereits drei Zeilen Code reichen aus, um ein Fenster zu erstellen:

```
import tkinter as tk    # importiert das Modul tkinter als tk
fenster = tk.Tk()       # erzeugt ein Fenster
fenster.mainloop()      # startet die Ereignisschleife
```

Das Hauptfenster muss aber erst mit einigen Befehlen individualisiert werden: Um das Erscheinungsbild anzupassen, stehen Methoden wie title(), iconbitmap(), geometry() und weitere zur Verfügung (Paz, 2018, S. 97-98).

```

import tkinter as tk    # importiert das Modul tkinter als tk
fenster = tk.Tk()        # erzeugt ein Fenster
fenster.title("jk 2025") # setzt den Fenstertitel
fenster.geometry("300x200") # setzt die Fenstergröße auf 300x200 Pixel
fenster.resizable(0, 0)  # verhindert, dass die Fenstergröße Userseitig
                          # verändert wird
fenster.state("normal")  # "zoomed" würde das Fenster in den Vollbildmodus
                          # versetzen
fenster.iconbitmap("favicon.ico") # setzt das Fenstericon
#fenster.destroy()       # schließt das Fenster
fenster.mainloop()       # startet die Ereignisschleife

```

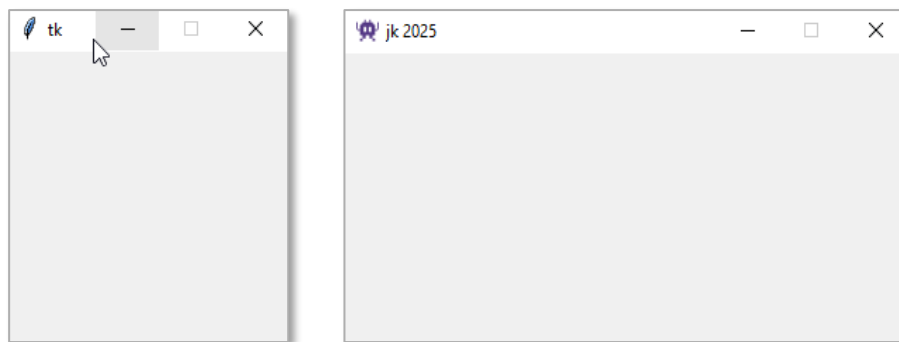


Abbildung 4 Vergleich zwischen einem Default-Fenster und nach ersten Anpassungen

Tkinter bietet eine große Zahl an Widgets, die als Grundbausteine für die Benutzeroberfläche dienen: Labels, Entries, Buttons etc. (siehe dazu Kapitel 4.3.1.2).

Mit Tkinter erstellte Anwendungen zeichnen sich durch hohen Bedienkomfort aus, da sie zu Benutzerinteraktionen visuelle Rückmeldungen geben und sich Widgets durch Layout-Manager dynamisch und flexibel im Fenster anordnen, selbst dann, wenn die Fenstergröße während der Laufzeit verändert wird (Bonacina, 2018, S. 192-193).

Ein grundlegender Aspekt von Tkinter ist das event-gesteuerte Programmiermodell. Das bedeutet konkret, dass Benutzeraktionen wie Klicks, Tastatureingaben oder Mausbewegungen Reaktionen auslösen. Die Aufgabe des Entwicklers ist es, Callback-Funktionen zu definieren, die jeweils an die entsprechenden Ereignisse (Events) gebunden werden. So entsteht eine interaktive und responsive GUI-Anwendung. Viele Eingaben werden bzw. können nur über bereits vorgegebene Optionen (wie beim

Button oder Radiobutton) erfolgen, was Fehler bei der Eingabe reduziert (Weigend, 2019, S. 431).

Auf die Argumente für die Entscheidung für Tkinter gegenüber anderen GUI-Frameworks wie beispielsweise PyQt, Kivy oder wxPython wurde bereits unter Punkt 4.2.1 eingegangen, ergänzend ist jedoch zu erwähnen, dass sich Tkinter insbesondere bei der schnellen Prototypentwicklung und durch die leichte Wartbarkeit des Codes hervortut (Theis, 2024, S. 339). Die API von Tkinter ist übersichtlich und leicht zu erlernen, was besonders für Einsteiger in diese Thematik von Vorteil ist (Paz, 2018, S. 31). Durch die Integration in die Python-Standardbibliothek erspart man sich zusätzlich Installationen und Tkinter-Anwendungen laufen problemlos unter Windows, Linux und macOS (Bonacina, 2018, S. 192). Überdies läuft Tkinter äußerst ressourcenschonend, wodurch es sich auch für Systeme mit begrenzten Ressourcen eignet (Moore A. D., 2021, S. 31).

Tkinter bietet eine solide Grundlage für die Entwicklung grafischer Benutzeroberflächen. Anwendungen lassen sich schnell entwickeln, testen, anpassen und erweitern, weshalb es die passende Wahl für die Konzeption und Entwicklung von Prototypen im Rahmen dieses Projektes ist.

In weiterer Folge soll näher auf die unterschiedlichen Layout-Manager, sowie Widgets und Benutzereingaben durch Mausereignisse eingegangen werden:

4.3.1.1 Layout-Manager in Tkinter – ein Überblick

Benutzeroberflächen werden in Tkinter mithilfe sogenannter Layout-Manager organisiert, diese steuern die Anordnung der Widgets in einem Fenster. Tkinter stellt dazu drei Hauptmethoden zur Verfügung: *place()*, *pack()* und *grid()*. Diese sollen näher betrachtet und ihre Stärken und Schwächen abgewogen werden, um eine bessere Entscheidungsgrundlage zu erhalten, welche Methoden für das Projekt infrage kommen bzw. welche für die Projektzwecke am geeignetsten sind.

Beim Layout-Manager **place()** werden Widgets anhand absoluter (oder auch relativer) Koordinaten im darüberliegenden Frame positioniert. Dabei werden die x-

und y-Werte explizit angegeben, sodass die Position eines Widgets in Bezug auf die linke obere Ecke des übergeordneten Container-Widgets festgelegt wird (Theis, 2024, S. 369).

Im Gegensatz dazu werden Widgets beim Einsatz des **pack()**-Managers nacheinander entlang einer bestimmten Seite (oben, unten, links oder rechts) im Frame angeordnet. Der insgesamt verfügbare Platz wird automatisch auf die Widgets aufgeteilt, sie werden in der Reihenfolge ihres Pack-Aufrufs positioniert (Weigend, 2019, S. 463-464).

Der **grid()**-Manager hingegen basiert auf dem Konzept eines Rasters, in dem Widgets in Zeilen und Spalten angeordnet werden. Dabei werden die Positionen durch die nullbasierten Indices der Parameter *row* und *column* festgelegt. Mit der Option *sticky* kann zudem bestimmt werden, an welchem Rand eines Rasters die Widgets ausgerichtet werden sollen (Weigend, 2019, S. 467).

	Vorteile	Nachteile
place()	• Absolute Kontrolle über die Positionierung • exakte Platzierung • relative Positionierung möglich (bei veränderlichen Fenstern von Vorteil) (Theis, 2024, S. 371)	• Die finale Größe des Fensters wird nicht automatisch an Inhalt angepasst, sondern muss explizit definiert werden • Widgets verlieren u. U. bei Änderungen der Fenstergröße ihre relative Position • Die manuelle Steuerung ist sehr aufwändig und fehleranfällig – insbesondere bei komplexen Layouts (Theis, 2024, S. 369–371)
pack()	• Einfache Handhabung, • ideal für einfache Layouts oder schnelle Prototypentwicklung • Automatische Anpassung der Anordnung, (Weigend, 2019, S. 464–465)	• Eingeschränkte Flexibilität • Die Reihenfolge der pack()-Aufrufe ist entscheidend, • Bei komplexen Benutzeroberflächen sind u. U. zusätzliche Frames notwendig (Weigend, 2019, S. 464–465)
grid()	• Das Rasterlayout ermöglicht eine präzise und strukturierte Anordnung von Widgets • Größe der Zellen passt sich automatisch an den Inhalt an • Reihenfolge der grid()-Aufrufe kann beliebig gewählt werden • Kombination von Frames und grid() erlaubt sehr komplexe und dynamische Benutzeroberflächen (Weigend, 2019, S. 467)	• Rasterlayout kann mit steigender Zahl von Spalten und Zeilen komplex wirken • Die optimale Verteilung von Widgets erfordert eine gewisse Planung (Weigend, 2019, S. 467)

Tabelle 6 Gegenüberstellung von Layout-Managern

Aus Tabelle 6 geht bereits hervor, dass der Layout-Manager `grid()` sehr viele, sehr gewichtige Vor- und im Gegenzug wenige, eher zu vernachlässigende Nachteile aufweist. `Grid()` bietet einen idealen Kompromiss zwischen Flexibilität und Gestaltungsmöglichkeiten. Der `place()`-Manager, mit der Notwendigkeit, feste Koordinaten manuell zu definieren, ist für kleinere Anwendungen oder Nischen besser geeignet, wohingegen `pack()` zwar für rasche Prototypenentwicklung geeignet ist, sich bei größeren Projekten mit der Notwendigkeit einer klaren Struktur und Übersichtlichkeit jedoch als unhandlich erweist (Weigend, 2019, S. 467). Das Rasterdesign von `grid()` fördert einen modularen – und somit besser wartbaren – Aufbau der Anwendung, wobei der Layout-Manager dem Entwickler die Arbeit abnimmt, sich um die exakte Reihenfolge oder Position der Widgets zu kümmern. Aufgrund dieser Vorteile – seiner Flexibilität und Benutzerfreundlichkeit – ist `grid()` die bevorzugte Methode, wenn es darum geht, komplexe Benutzeroberflächen zu erstellen (Steyer, 2018, S. 236–237), somit wird auch im Rahmen der vorliegenden Arbeit mit `grid()` gearbeitet.

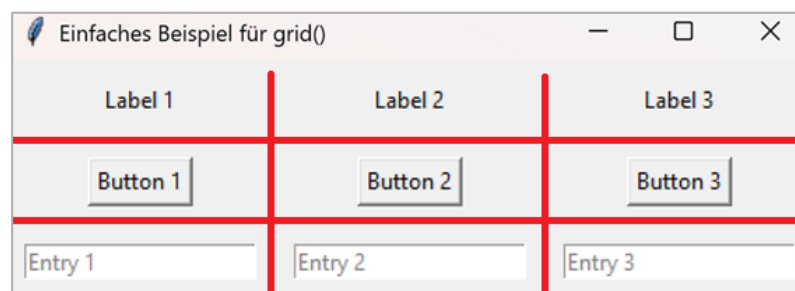


Abbildung 5 Veranschaulichung der Arbeitsweise des Layout-Managers `grid()`; hier in einer 3x3 Geometrie

Zu erwähnen gilt es allerdings, dass innerhalb eines Projekts unterschiedliche Layout-Manager zum Einsatz kommen können, etwa wenn die Container-Widgets in einem `grid()`-Raster angeordnet werden, die darin enthaltenen Widgets wie z. B. Buttons innerhalb des Containers aber etwa mit `pack()` oder `place()` angeordnet werden.

4.3.1.2 Widget Typen

Tkinter stellt eine große Zahl von Widgets zur Verfügung, die für die Erstellung einer grafischen Benutzeroberfläche genutzt werden können. Jedes erfüllt eine ganz spezifische Funktion und bietet individuelle Konfigurationsmöglichkeiten. In diesem Kapitel werden die wichtigsten Widgets erläutert, die für das Projekt relevant sind.

Das **Frame** ist das grundlegendste Widget, es dient als Parent-Widget zur Gruppierung einzelner oder mehrerer untergeordneter Widgets fungiert und so das wesentlichste Werkzeug zur Strukturierung der GUI ist (Chandrakar & Bahadure, 2023, S. 232).

```
frame = tk.Frame(width=300, height=200, borderwidth=2, relief="groove")
```

Ein **Label** dient zur Anzeige von Text oder Bildern innerhalb der GUI. Es kann für Beschriftungen, Statusanzeigen oder Hinweise verwendet werden (Chandrakar & Bahadure, 2023, S. 201). Über die Attribute *text*, *font*, *foreground* und *borderwidth* können Anpassungen vorgenommen werden (Weigend, 2019, S. 440).

```
label = tk.Label(root, text="ein Label", font=("Helvetica", 12), fg="green")
```

Ein **Button**-Widget ist ein einfacher Druckknopf, der bei Betätigung eine Callback-Funktion aufruft (Moore A. D., 2021, S. 108).

```
exit_button = tk.Button(root, text="Beenden", command=root.quit)
```

Als Eingabefelder für den Benutzer dienen die Widgets **Entry**, **Text** und **ScrolledText**. Das **Entry**-Widget ermöglicht die Eingabe von Text oder Zahlen durch den Benutzer. Über den Parameter *show* = „*“ wird die versteckte Eingabe ermöglicht, anstelle des tatsächlich eingegebenen Zeichens wird ein * angezeigt. Größere Textmengen hingegen können mit dem **Text**-Widget eingegeben werden. **ScrolledText** bietet eine integrierte Scrollleiste, die den Text besser überblicken lässt (Theis, 2024, S. 343, 350). Für das Projekt sind insbesondere das Entry-Widget – für Benutzereingaben – und das ScrolledText-Widget – für die Ausgabe von Messergebnissen und ein Textlog – von Nutzen.

```
default_entry = tk.Entry(root)
password_entry = tk.Entry(root, show="*")

text_widget = tk.Text(root, height=10, width=50)
text_widget.insert(tk.END, "Das ist ein Text-Widget.\nHier kann man mehrere  
Zeilen schreiben.")
```

```

scrolled_text_widget = scrolledtext.ScrolledText(root, height=10, width=50,
wrap=tk.WORD)
scrolled_text_widget.insert(tk.END, "Das ist ein ScrolledText-Widget.\nHier
kann man mehrere Zeilen schreiben und scrollen.

```

Für die **Auswahl von Elementen** stehen **Listbox**, **Spinbox** und **Combobox** zur Verfügung. Das **Listbox**-Widget erlaubt die Auswahl eines oder mehrerer Elemente aus einer Liste. Eine **Spinbox** hingegen ermöglicht die Auswahl vordefinierter Werte durch Erhöhen oder Verringern des Nummern-Wertes (Theis, 2024, S. 352, 354). Die **Combobox** ist eine erweiterte Variante der Spinbox, die eine Auswahl aus vorgegebenen Werten oder die Eingabe neuer erlaubt. Über den Parameter *state=„readonly“* können individuelle Nutzereingaben ausgeschlossen werden (Meier, 2017, S. 86).

```

listbox = tk.Listbox(root)
beispiel_liste = ["Eintrag 1", "Eintrag 2", "Eintrag 3"]
for item in beispiel_liste:
    listbox.insert(tk.END, item)

spinbox = tk.Spinbox(root, from_=0, to=10)
spinbox.pack()

combobox = ttk.Combobox(root, values=["Option 1", "Option 2", "Option 3"])

```

Das Listbox-Element eignet sich zur Darstellung des geplanten Messablaufs, während die Combobox beispielsweise bei der Auswahl der Baudrate im Projekt zur Anwendung kommen kann.

Als **Options-Widgets** bietet Tkinter **Radiobuttons** und **Checkboxes**. **Radiobuttons** sind ein allgemein bekanntes Element der GUI-Programmierung. Mit ihrer Hilfe kann der Benutzer eine von mehreren Optionen wählen (Theis, 2024, S. 357). **Checkbuttons** funktionieren nach einem ähnlichen Prinzip, erlauben aber Mehrfachauswahl (Theis, 2024, S. 357, 361). Die Widgets speichern ihre Zustände per default in einer booleschen Variablen. Mehrere dieser Widgets können (bzw. im Fall von Radiobuttons: müssen) mit einer *StringVar*-Kontrollvariablen verknüpft werden (Weigend, 2019, S. 447).

```
radio_var = tk.StringVar(value="Option 1")
radio_button1 = ttk.Radiobutton(root, text="Option 1", variable=radio_var,
value="Option 1", command=on_radio_button_selected)

checkbox_var = tk.BooleanVar()
checkbox1 = ttk.Checkbutton(root, text="Checkbox 1", variable=checkbox_var1,
command=on_checkbox_toggled)
```

Das Widget **Scale** ist ein Schieberegler, über den Werte justiert werden können: das Widget bietet sich an, um Zahlenwerte durch den Nutzer einzugeben (Theis, 2024, S. 363).

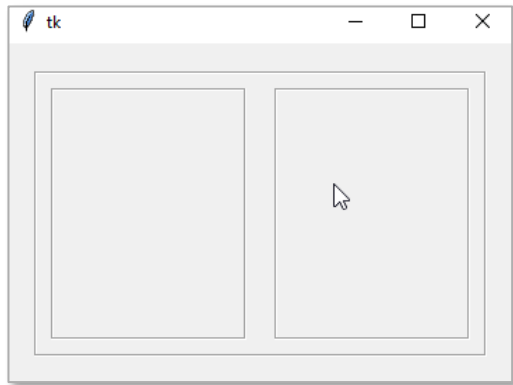
```
scale = tk.Scale(root, from_=0, to=100, orient=tk.HORIZONTAL,
command=show_value)
```

Eine **Scrollbar** ist eine Scrollleiste, die beispielsweise an *Text*- oder *Listbox*-Widgets angehängt werden kann, um die Inhalte scrollbar zu machen, was die Übersicht und somit die Benutzerfreundlichkeit erhöht, wenn eine große Menge an Text bzw. Informationen abgebildet wird (Theis, 2024, S. 352).

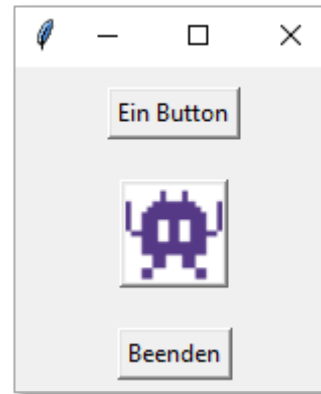
```
listbox_frame = ttk.Frame(root)
listbox_scrollbar = ttk.Scrollbar(listbox_frame, orient=tk.VERTICAL)
listbox_widget = tk.Listbox(listbox_frame,
yscrollcommand=listbox_scrollbar.set)
listbox_scrollbar.config(command=listbox_widget.yview)
```

Mit dem Canvas-Widget können Zeichnungen und Diagramme dargestellt werden, so unter anderem Linien, Rechtecke oder Kreise (Chandrakar & Bahadure, 2023, S. 282). Ein Canvas eignet sich ganz besonders zur Visualisierung von Messdaten, die über den CAN-Bus übertragen werden. Der Vollständigkeit halber wird dieses Widget an dieser Stelle aufgelistet, näher wird jedoch im Kapitel 4.3.2.5 darauf eingegangen.

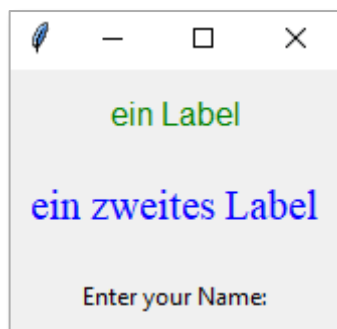
Mehrere verschachtelte Frames, die Anzeige der Rahmen ist optional:



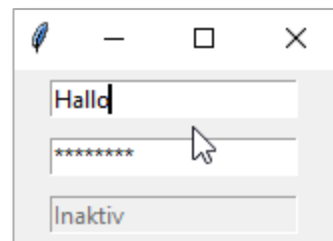
Unterschiedliche Buttons; der Text kann durch ein Bild ersetzt werden:



Label dienen der Anzeige von Text (oder Bildern) und können individuell angepasst werden.



Entryfelder nehmen einzeilige Benutzereingaben entgegen:



Für mehrzeilige Benutzereingaben gibt es das Text- und das ScrolledText-Widget an:

Eine Auswahl ermöglichen die Listbox, Spinbox und Combobox:

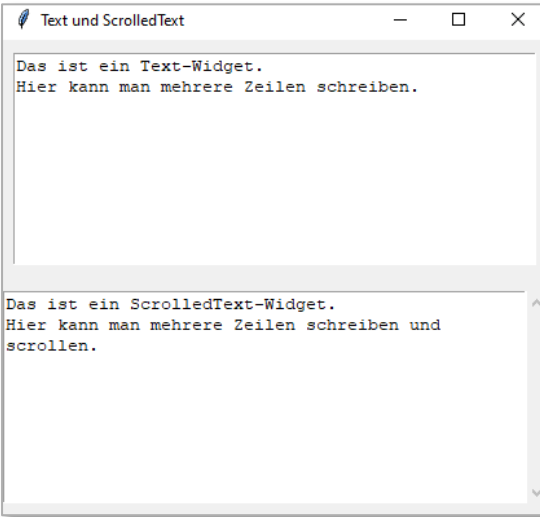
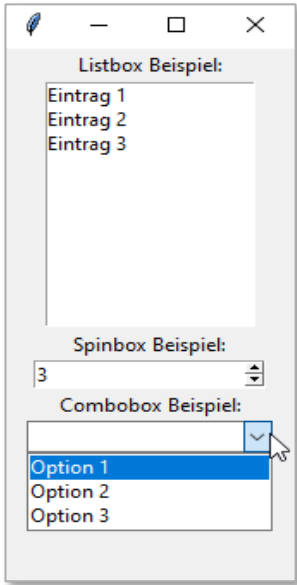
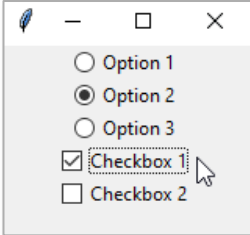
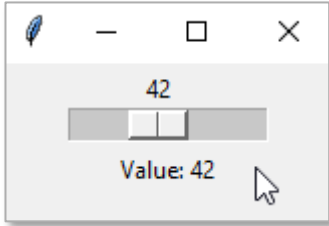
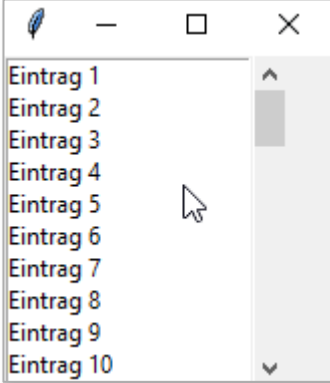
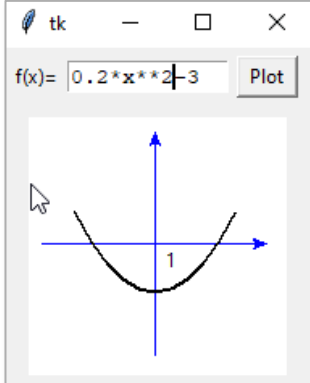
	
Radiobuttons und Checkboxes erlauben die Auswahl aus vordefinierten Optionen:	Mit der Scale können numerische Werte eingestellt werden:
	
Eine Scrollbar macht Listboxes scrollbar:	Canvas: Visualisierung von Daten
	

Tabelle 7 Exemplarische Veranschaulichung der relevanten Widget-Typen

4.3.1.3 Mausereignisse

Interaktionen des Benutzers mit der Anwendung über die Maus spielen eine wichtige Rolle in GUI-Applikationen. Jedes Widget kann grundsätzlich mit Mausereignissen verknüpft werden.

Die wichtigsten Mausereignisse sind:

- **Button1** und **Button3** bzw. **ButtonRelease1** und **ButtonRelease3**: Drücken bzw. Loslassen der linken oder rechten Maustaste
- **Enter**, **Motion** und **Leave**: Bewegung der Maus in ein Widget hinein, innerhalb eines Widgets oder aus einem Widget hinaus (Theis, 2024, S. 367)

Die Verknüpfung des Mausereignisses mit einer Callback-Funktion erfolgt über die Methode `bind()`. Das damit übergebene Event-Objekt enthält zusätzliche Informationen, wie beispielsweise die Position der Maus als x- und y-Koordinate relativ zur linken oberen Ecke des Anwendungsfensters (Theis, 2024, S. 369)

```
[WIDGET].bind([AKTION], [AUFZURUFENDE FUNKTION])
```

4.3.2 Weiterführende Funktionen & Konzepte

Darüber hinaus bietet Tkinter eine beinahe unerschöpfliche Reihe tiefergehender Konzepte, die eine individuelle Anpassung der GUI-Anwendung erlauben. So können Widgets in ihrer Darstellung und ihrem Verhalten durch verschiedene Parameter (Farbgebung, Schriftarten, Ändern des Mauszeigers etc.) verändert werden. Unter anderem können – für ein konsistenteres Erscheinungsbild – für bestimmte Widgets Standardwerte festgelegt werden. Das Text-Widget erlaubt überdies Formatierungsfunktionen, was die Logausgaben übersichtlicher gestaltet (Paz, 2018, S. 153-198). Auch eine dauerhaft sichtbare Menüleiste dient gleichermaßen der Benutzerfreundlichkeit der Anwendung wie an Widgets gekoppelte Kontextmenüs, die nur bei Bedarf aufscheinen (Theis, 2024, S. 375).

Auf einige, für das Projekt relevante Konzepte soll nachfolgend näher eingegangen werden:

4.3.2.1 Menüleisten

Menüleisten sind ohne Frage ein elementarer Teil einer gut strukturierten Benutzeroberfläche. Vorteilhaft an Tkinter ist, dass die Darstellung der Menüleiste sich stets dem jeweiligen Betriebssystem anpasst, unter dem die Anwendung ausgeführt wird (Paz, 2018, S. 228).

```
mLeiste = tkinter.Menu(fenster) #Menüleiste erzeugen
mDatei = tkinter.Menu(mLeiste) #Menüs erstellen
mDatei.add_command(label="Neu") #Menüpunkte hinzufügen
mLeiste.add_cascade(label="Datei", menu=mDatei) #Menüs zur Menüleiste
hinzufügen
fenster["menu"] = mLeiste #Menüleiste dem Fenster zuweisen
```

Wichtige Elemente zur Individualisierung sind dabei menüeigene Radiobuttons und Checkbuttons, die die Auswahl einer Option innerhalb einer Gruppe oder das Togglen bestimmter Einstellungen erlauben. Weitere Menüoptionen, die an dieser Stelle Erwähnung finden müssen, sind *add_separator()*, womit eine Trennlinie zwischen Menüpunkten eingefügt wird, und der Parameter *tearoff=0*. Mit dem Befehl *underline* kann festgelegt werden, welcher Buchstabe beim Drücken der ALT-Taste unterstrichen wird (Theis, 2024, S. 377-378).

```
mLeiste = tkinter.Menu(fenster) # Menüleiste erzeugen
mDatei = tkinter.Menu(mLeiste, tearoff=0) # Menüs erstellen mit tearoff=0
mDatei.add_command(label="Neu", underline=0) # Menüpunkte hinzufügen
mDatei.add_separator() # Separator hinzufügen
mDatei.add_radiobutton(label="Grün", variable=farbe, value="#58FA58",
command=lambda: fenster.config(bg=farbe.get()), underline=0)
mDatei.add_radiobutton(label="Blau", variable=farbe, value="#5882FA",
command=lambda: fenster.config(bg=farbe.get()), underline=0)
mDatei.add_separator() # Separator hinzufügen
mDatei.add_checkbutton(label="Rand", variable=rand, onvalue="solid",
offvalue="flat", command=lambda: fenster.config(relief=rand.get()),
underline=0)
mLeiste.add_cascade(label="Datei", menu=mDatei, underline=0) # Menüs zur
Menüleiste hinzufügen
fenster["menu"] = mLeiste # Menüleiste dem Fenster zuweisen
```

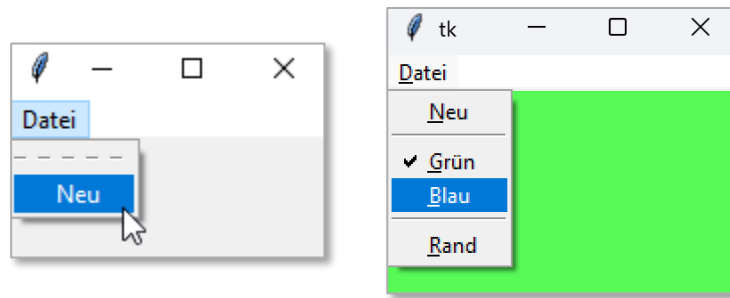


Abbildung 6 Eine rudimentäre Menüleiste und eine – mit den erwähnten Optionen – angepasste

4.3.2.2 Kontextmenüs und Dialogfelder in Tkinter

Auch für Kontextmenüs und Dialogfelder bietet Tkinter plattformunabhängige Mechanismen.

Bei einem Rechtsklick¹ auf ein Widget ermöglichen **Kontextmenüs** den schnellen Zugriff auf selbst definierte Zusatzfunktionen und Anpassungsmöglichkeiten. Kontextmenüs werden mit `add_command()` hinzugefügt, mit `bind()` an ein Widget gebunden und mit `tk_popup()` aufgerufen (Theis, 2024, S. 379). Die Implementierung von Kontextmenüs hat sich in der Prototypisierungsphase als nicht zielführend erwiesen und wurde im weiteren Verlauf verworfen, bleibt aber dennoch ein wichtiges Tool in der GUI-Entwicklung.

Dialogfelder hingegen sind ein fester Bestandteil der im Projekt erarbeiteten Anwendung geworden. Tkinter bietet verschiedene Standarddialogfelder (Warn-, Fehlermeldungen, Bestätigungsabfragen etc.). Die `messagebox`-Funktion bietet vordefinierte Dialoge für unterschiedliche Anwendungszwecke (Theis, 2024, S. 381, 386).

```
def yesno():
    antwort = mb.askyesno("Frage", "Möchten Sie fortfahren?")
    print("Antwort:", "Ja" if antwort else "Nein")
```

¹ Unter macOS werden Eingaben mit Maustasten anders verarbeitet, das Kontextmenü wird dort daher üblicherweise mit der linken Maustaste aufgerufen (Theis, 2024, S. 380).

Die Methoden *askyesno()* oder *askokcancel()* liefern boolesche Rückgabewerte, während *showinfo()*, *showwarning()* und *showerror()* nichts zurückgeben (Theis, 2024, S. 384).

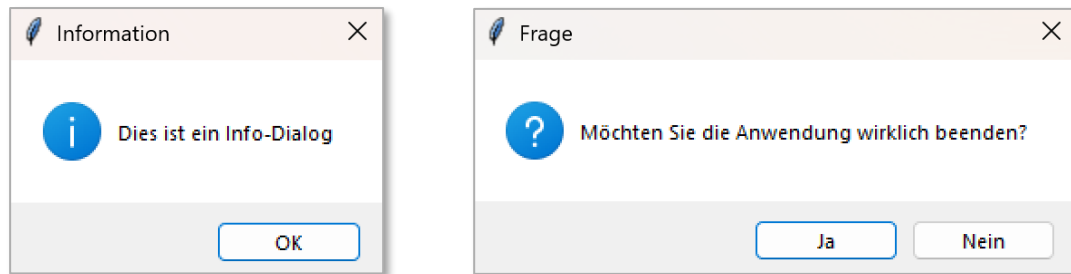


Abbildung 7 Beispiele für Dialogfelder

Von großer Wichtigkeit für das Projekt sind auch **Dateidialoge**, insbesondere wenn es um das Laden oder Speichern von Konfigurationen in eine Datei geht. Hierfür wurde das Modul `filedialog` verwendet, welches Funktionen zum Öffnen und Speichern von Dateien enthält. Die wichtigsten Funktionen lauten *askopenfilename()* und *asksaveasfilename()* zum Öffnen und Speichern von Dateien sowie *askdirectory()* zum Auswählen von Verzeichnissen (Chandrakar & Bahadure, 2023, S. 284).

4.3.2.3 Tabs in Tkinter-Fenstern

Tabs sind ein dienliches Mittel zur übersichtlichen Strukturierung von GUI-Anwendungen. Mit dem Notebook-Widget können verschiedene Inhalte logisch voneinander getrennt werden, insbesondere bei großen bzw. komplexen Anwendungen kommt dieser Vorteil besonders zum Tragen (Meier, 2017, S. 159). Es wird jedoch nicht näher darauf eingegangen, da diese Option im Laufe der Projektarbeit nicht mehr verwendet wurde.

4.3.2.4 Validierung von Benutzereingaben in Tkinter

Eine Funktion, die in ihrer Wichtigkeit nicht genug betont werden kann, ist die Validierung von Benutzereingaben, denn Tkinter bietet keine vorgefertigten Widgets für Benutzereingaben mit einem bestimmten Format. Die Überprüfung, ob die Eingabe beispielsweise den richtigen Datentyp enthält, muss manuell implementiert werden. Trotz des größeren Entwicklungsaufwands stellt dies mehr Gestaltungs- und Anpassungsmöglichkeiten zur Verfügung.

Es gibt dabei unterschiedliche Ansätze, um „falsche“ Eingaben zu handhaben: Ungültige Zeichen direkt bei der Eingabe blockieren, Fehlermeldungen bei Bestätigung durch den Nutzer oder Wechseln des Fokus („Focus-Out-Validierung“)², Korrektur fehlerhafter Eingaben (Moore A. D., 2021, S. 171-173). Die Implementierung erfolgt über den parameter *validate*:

- *validate="key"*: Validierung bei jeder Eingabe
- *validate="focusin"*: Validierung, wenn das Feld den Fokus erhält
- *validate="focusout"*: Validierung, wenn das Feld den Fokus verliert.
- *validate="all"*: Validierung in allen Fällen
- *validatecommand*: Möglichkeit, die Validierung von einer selbst geschriebenen Funktion mit Bool-Rückgabewert und selbst definierten Prüfkriterien durchzuführen

Dabei ist *validatecommand* die vorherrschende Methode in diesem Projekt. Nachfolgend ein Beispiel für die konkrete Umsetzung. Dabei ist auf die korrekte Reihenfolge zu achten:

```
def validate_can_id(P):
    if P.isdigit() and 0 <= int(P) <= 2047:
        return True
    elif P == "":
        return True
    else:
        return False
vcmd = (root.register(validate_can_id), '%P')
can_id_entry = ttk.Entry(initialize_frame, textvariable=can_id_var,
validate='key', validatecommand=vcmd, width=10)
```

4.3.2.5 Datenvisualisierung mit Canvas-Objekten in Tkinter

Die Ausgabe der Messdaten in einer grafischen Darstellung war von Beginn an ein Ziel, das mit diesem Projekt erreicht werden sollte. Die Canvas-Klasse bietet die nötigen Werkzeuge dafür. Linien, Rechtecke, Ovale oder Texte können beliebig platziert und auch animiert werden, um Dynamiken abzubilden (Weigend, 2019, S. 491).

² Mit Fokus ist in diesem Zusammenhang gemeint, dass ein Widget aktiv ausgewählt ist.

Als Zeichenmethoden dienen die folgenden:

<code>create_line()</code>	Zeichnet eine Linie, optional mit Pfeilspitzen
<code>create_rectangle()</code>	Zeichnet ein Rechteck; Rahmen- und Füllfarbe sind anpassbar
<code>create_oval()</code>	Zeichnet ein Oval/einen Kreis
<code>create_polygon()</code>	Zeichnet ein Vieleck
<code>create_text()</code>	Fügt ein Textelement ein

Tabelle 8 Zeichenmethoden im Canvas (Theis, 2024, S. 388-389)

Mit der Methode *move()* können Canvas-Objekte verschoben werden. Mit der *after()*-Methode kann eine *move()*-Funktion wiederholt aufgerufen werden, was einen Animationseffekt erzeugt. Die Kollisionserkennung wird mit *find_overlapping()* realisiert (Theis, 2024, S. 389-392). Darüber hinaus können auch komplexe Diagramme, wie Kreisdiagramme oder Funktionsgraphen, erstellt werden (Weigend, 2019, S. 496).

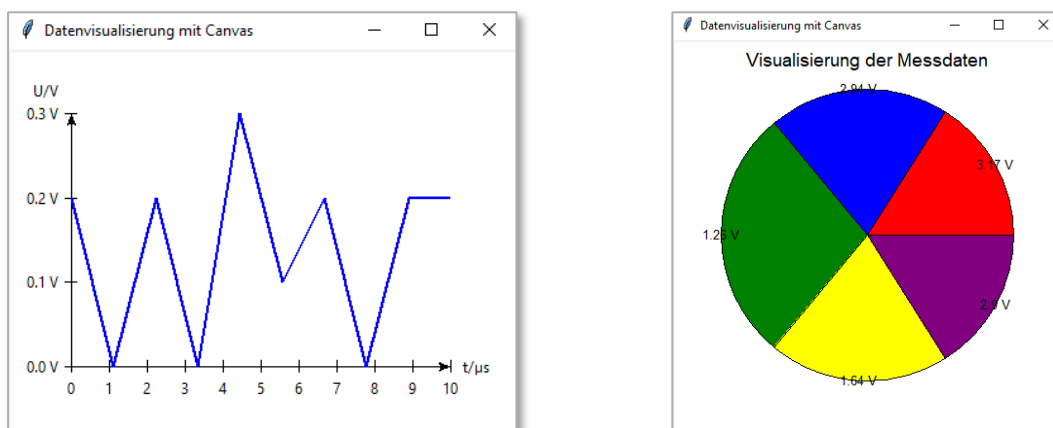


Tabelle 9 Frühe Beispiele für Datenvisualisierung aus der Anfangsphase des Projekts

4.4 Softwarearchitektur & Softwareoptimierung

In diesem Kapitel wird auf die grundlegenden Säulen eines guten Programmierstils eingegangen. Letztlich sollte die zu entwickelnde Anwendung nicht nur funktional, sondern auch gut lesbar, erweiterbar sowie gut wartbar sein.

4.4.1 Design- und Programmierprinzipien nach PEP 8

Für Python gibt es den *Style Guide for Python Code* (PEP 8), der Konventionen für eine einheitliche Codegestaltung vorgibt (van Rossum, Warsaw, & Coghlan, 2013).

Dabei handelt es sich zwar nicht um zwingende Vorgaben, sondern um Empfehlungen, und in bestimmten Fällen kann es sinnvoll sein, davon abzuweichen. Wichtig ist jedoch immer, dass der Code konsistent und einheitlich gestaltet ist (Knoll & Heedt, 2022, S. 574). Denn insbesondere bei Teamarbeiten ist eine einheitliche Formatierung wichtig, weil unterschiedliche Stile die Arbeit der einzelnen Entwickler deutlich erschweren können. Es gibt jedoch auch Werkzeuge, die dabei helfen, die Formatierungsregeln einzuhalten – Black („The Uncompromising Code Formatter“) ist wohl das bekannteste davon (Sweigart, 2022, S. 68).

Grundlegende Empfehlungen aus der PEP 8 sind etwa eine Einrückung von 4 Leerzeichen pro Ebene, beidseitig einschließende Leerzeichen um Operatoren (+, -, == etc.), die Begrenzung der Zeilenlänge auf maximal 79 Zeichen und zwei Leerzeilen zwischen Klassen und Funktionen bzw. eine Leerzeile zwischen Klassen-Methoden. Als Namenskonventionen werden *snake_case* für Funktionen und Variablen vorgeschlagen, *PascalCase* für Klassennamen und *SNAKE_CASE* für Konstanten. Für Variablen sollen aussagekräftige Namen gewählt werden, denn kurze oder allgemein gehaltene Namen wie „a“, „var“ oder „num“ geben keinen Aufschluss über deren Verwendung, während Bezeichner wie „email_address“ oder „monthly_payment“ bereits einen deutlichen Eindruck von ihrem Inhalt geben (Sweigart, 2022, S. 69, 76).

4.4.2 Modularer Aufbau

Ein modularer Aufbau des Codes erleichtert die Arbeit damit erheblich. Wichtige Methoden zur Modulierung sind Funktionen, Module und Objektorientierung (OOP). Während Python nicht strikt objektorientiert aufgebaut ist, bieten Klassen und Objekte eine sinnvolle Strukturierungsmöglichkeit für GUI-Anwendungen.

Funktionen agieren wie Blackboxes und ermöglichen so eine klare Trennung von Programmteilen. So wird der Codeumfang reduziert, indem Redundanzen vermieden werden. Mehrmals verwendeter Code muss nur einmal definiert werden, was die

Anwendung übersichtlicher macht. Python erlaubt jedoch im Gegensatz zu anderen Sprachen keine Funktionsüberladung – nur die zuletzt erfolgte Definition gilt und überschreibt alle vorangegangenen (Theis, 2024, S. 79). Funktionen können auch mehrere Rückgabewerte liefern, indem sie ein Tupel zurückgeben (Theis, 2024, S. 178). Eine besonders wichtige Technik ist die Verwendung von **args* und ***kwargs*, womit sich eine flexible Anzahl von Argumenten übergeben lässt (Schäfer, 2019, S. 42). Spezielle built-in Funktionen wie *map()* und *filter()* erleichtern Iterationen über große Datenmengen. Während *map()* die Mehrfachausführung einer selbst geschriebenen Funktion mit jeweils anderen Parametern ermöglicht, nimmt *filter()* ein Container-Objekt entgegen und liefert als Rückgabewert ein Objekt, das diejenigen Elemente enthält, für die eine selbst definierte Funktion den booleschen Wert *True* zurückgibt (Theis, 2024, S. 162–164).

Module erlauben es, Teile des Codes (z.B. Funktionen und Klassen) in separate Dateien auszulagern, wodurch der Code übersichtlicher und sogar in anderen Programmen wiederverwendbar wird. Der Import gelingt wie folgt:

```
import [modulname]           # Standardimport
import [modulname] as [alias] # Import mit Alias
from [modulname] import [function] # Einzelne Funktionen importieren:
from [modulname] import *      # Gesamtes Modul importieren; nicht
                               # empfohlen wegen möglicher Namenskonflikte
```

(Theis, 2024, S. 202-203)

Modulnamen dürfen allerdings keine Punkte enthalten, auch auf die Groß- und Kleinschreibung ist zu achten (Ernesti & Kaiser, 2023, S. 495). Wichtige Standardmodule, die auch im Zuge dieses Projekts zum Einsatz kommen, sind *math* und *cmath* (für mathematische Funktionen), *os* (als Schnittstelle zum Betriebssystem; Dateiverwaltung) und *sys* (für den Zugriff auf Kommandozeilenargumente) (Schäfer, 2019, S. 46-49).

In Python wird nicht streng **objektorientiert** programmiert. GUI-Anwendungen bilden hier üblicherweise eine Ausnahme, weil OOP oft zur Strukturierung genutzt wird. Bereits das Toplevel-Fenster in einer Tkinter-Anwendung wird als Objekt der Klasse *TK()* erstellt:

```
root = Tk()
```

Diese Klasse enthält alle weiteren GUI-Elemente wie Label, Button oder Frames. Dabei wird zwischen öffentlichen („public“), geschützten („protected“) und privaten („private“) Klassen-Memberrn unterschieden. Private Attribute beginnen mit `__` und sind, wie auch in anderen Sprachen, nur innerhalb der Klasse zugänglich (Steyer, 2018, S. 157-158).

4.4.3 Fehler/Ausnahmen im Programmcode und deren Behandlung

Fehler sind leider ein unvermeidlicher Bestandteil der Softwareentwicklung. In Python gibt es drei Hauptkategorien: Syntaxfehler, Laufzeitfehler und logische Fehler (Ulias, 2024, S. 108).

Syntaxfehler entstehen bereits beim Einlesen des Codes und verhindern eine Ausführung des Programms, häufige Ursachen sind fehlende Doppelpunkte, falsche Einrückungen oder ungültige Zeichen (Downey, 2021, S. 251). Mit der Versionsnummer 3.10 wurden Fehlermeldungen in Python systematisch verbessert, um hilfreichere Hinweise und Korrekturvorschläge zu geben (Theis, 2024, S. 74).

Laufzeitfehler treten hingegen während der Programmausführung auf, eine Division durch Null oder ein Zugriff auf eine nicht existierende Datei können einen Absturz auslösen. Zur Vermeidung werden kritische Codepassagen mit try-except-Blöcken abgefangen (Knoll & Heedt, 2022, S. 517):

```
try:
    zahl = int(input("Bitte eine ganze Zahl eingeben: "))
except ValueError:
    print("Fehler: Keine gültige Zahl eingegeben.")
```

Logische Fehler führen dazu, dass ein Programm unerwartete oder falsche Ergebnisse liefert, obwohl es ansonsten fehlerfrei ausgeführt wird. Sie sind schwer zu erkennen und erfordern Techniken zum Debugging wie Einzelschrittverfahren oder Logging (Downey, 2021, S. 257).

Das **Modul *logging*** bietet Funktionen zum Aufzeichnen von Fehlermeldungen und Ereignissen, um den Programmablauf besser nachzuvollziehen. Mit verschiedenen Log-Leveln (DEBUG, INFO, WARNING, ERROR, CRITICAL) kann die Ausgabe der Fehlermeldungen gesteuert werden (Sweigart, 2020, S. 312-316).

Viele Entwicklungsumgebungen (IDEs) bringen **Debugging-Werkzeuge** mit, die Variablen überwachen, Breakpoints setzen und Code schrittweise ausführen können (Ulias, 2024, S. 108). Möglich ist auch die Ausgabe von Variablenwerten zu einem bestimmten Zeitpunkt mit `print()`. Davon wird allerdings ausdrücklich abgeraten (Sweigart, 2020, S. 315).

Mit *raise* können gezielt Fehler ausgelöst werden, um fehleranfällige Codeabschnitte zu testen (Theis, 2024, S. 172).

4.5 Kommunikation, Verarbeitung und Ausgabe der Messdaten

4.5.1 CAN-Bus: Nachrichten schreiben und auslesen

Bevor Daten übermittelt werden können, muss das CAN-Interface mit der Funktion *CAN_Initialize* initialisiert werden. Dadurch wird der Transceiver betriebsbereit gemacht. Mit dem Befehl *CAN_Write* wird eine Nachricht auf den CAN-Bus geschrieben, mit *CAN_Read* wird sie gelesen. Über Filterfunktionen können beispielsweise nur bestimmte CAN-IDs gelesen werden, ereignisgesteuerte Kommunikation verkürzt die Reaktionszeit. Nach Abschluss der Kommunikation wird der Bus mit *CAN_Uninitialize* wieder freigegeben. Die Kommunikation über CAN beruht auf dem Austausch von Datenpaketen, sogenannten CAN-Frames, die eine CAN-ID sowie die Nutzdaten und Steuerbits enthalten. Über die API der Firma PEAK System GmbH erfolgen das Empfangen und Senden von Nachrichten. (Peak System Technik GmbH, 2024, S. 30-31).

```
pcan = PCANBasic() # Instanz der Klasse PCANBasic erstellen
status = pcan.Initialize(PCAN_USBBUS1, PCAN_BAUD_500K) # Initialisierung (500
kBit/s)

if status != 0: # erfolgreiche Initialisierung prüfen
    print(f"Fehler beim Initialisieren des PCAN-Interfaces: {status}")
```

```

else:
    print("PCAN-Interface erfolgreich initialisiert")

# Nachricht senden
msg = TPCANMsg() # Instanz der Klasse TPCANMsg erstellen (Nachricht)
msg.ID = 0x100 # CAN-ID
msg.MSGTYPE = PCAN_MESSAGE_STANDARD # Standard-CAN-Nachricht
msg.LEN = 8 # Länge der Nachricht
msg.DATA = (0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08) # Beispiel-Daten

send_status = pcan.Write(PCAN_USBBUS1, msg) # Nachricht senden, Status
speichern
if send_status != 0:
    print(f"Fehler beim Senden der Nachricht: {send_status}")
else:
    print("Nachricht erfolgreich gesendet")

# Nachricht empfangen
read_status, received_msg, timestamp = pcan.Read(PCAN_USBBUS1) # Nachricht
empfangen und Status speichern
if read_status == 0:
    print(f"Empfangene Nachricht - ID: {received_msg.ID}, Daten:
{list(received_msg.DATA)}") # Empfangene Nachricht ausgeben (ID und Daten)
else:
    print(f"Fehler beim Empfangen der Nachricht: {read_status}") #
Fehlermeldung ausgeben (falls vorhanden)

# PCAN-Interface freigeben
pcan.Uninitialize(PCAN_USBBUS1) # PCAN-Interface freigeben (USB-BUS1)
print("PCAN-Interface freigegeben")

```

Dabei wird die Schnittstelle mit einer Baudrate von 500 kbit/s initialisiert, und eine CAN-Nachricht mit der ID „0x100“ und 8 Byte Nutzdaten gesendet. In weiterer Folge wird eine Nachricht über den Bus empfangen und letztlich wird der Bus wieder freigegeben.

Mit der API wird die PCAN-Basic-Bibliothek bereitgestellt, die zusätzlich viele Funktionen bietet, etwa *PCAN_CHANNEL_CONDITION*, um anzuzeigen, ob dieser Kanal verfügbar ist. *PCAN_LISTEN_ONLY* setzt das Gerät in einen Wartemodus, wo keine Nachrichten gesendet, sondern nur empfangen werden. Mit *PCAN_RECEIVE_EVENT* werden Event-Triggern für eingehende Nachrichten erstellt (Peak System Technik GmbH, 2024a, S. 8-9, 45-56, 53-55).

4.5.2 Speicherung der Messdaten/Konfigurationen in einer Datei

Eine zentrale Aufgabe der zu erstellenden Anwendung ist die Speicherung und das Abrufen von Mess- und Konfigurationsdaten in bzw. aus Dateien. Python bietet dafür unterschiedliche Methoden, wie das Schreiben in txt- oder CSV-Dateien, aber auch in binären Formaten wie Pickle und JSON (Ernesti & Kaiser, 2023, S. 138).

Um eine Datei zu bearbeiten, wird sie mit der Funktion *open()* geöffnet. Dateiname und Zugriffsmodus werden dabei als Parameter übergeben, die wichtigsten Modi sind *r* (lesen, Defaultwert), *w* (schreiben), *a* (anhängen von Inhalten ans Ende der Datei), *r+* (lesen und schreiben mit aktueller Position) und *w+* (lesen und schreiben, Datei wird zuvor geleert) (Theis, 2024, S. 273-274). Zwingend notwendig ist auch das Schließen der Datei mit *close()*, nachdem alle Daten geschrieben oder gelesen wurden, damit die Datei wieder freigegeben wird (Ernesti & Kaiser, 2023, S. 140).

Die Speicherung von Daten in Textdateien ist einfach und leicht lesbar, allerdings weniger performant als binäre Formate:

```
with open("messdaten.txt", "w") as datei:
    datei.write("Zeit;Messaufbau;Spannung\n")
    datei.write("10:00;001;3.7\n")
```

Durch das *with*-Statement wird die Datei nach dem Lesen oder Schreiben automatisch geschlossen.

Mit *readlines()* werden die im txt-File gespeicherten Werte als Liste eingelesen, ein zeilenweises Einlesen erfolgt hingegen mit *readline()* (Theis, 2024, S. 277).

CSV-Dateien sind besonders geeignet für strukturierte Datensätze. Beim Schreiben einer CSV-Datei müssen Zahlen korrekt formatiert werden, denn Beistriche (oder Semikolons) sind für die Gliederung der Daten reserviert. Eingelesen werden Daten mit *csv.reader()*, was eine Liste als Rückgabewert hat (Theis, 2024, S. 280-282).

```
import csv
with open("messdaten.csv", "w", newline="") as datei:
    writer = csv.writer(datei, delimiter=";")
    writer.writerow(["Zeit", "Messaufbau", "Spannung"])
    writer.writerow(["10:00", "001", "3.7"])

with open("messdaten.csv", "r") as datei:
```

```
reader = csv.reader(datei, delimiter=";")
for row in reader:
    print(row)
```

Eine effizientere Art der Speicherung wird mit dem **pickle-Modul** umgesetzt. Daten werden dabei in binärer Form gespeichert, was ein schnelleres Einlesen erlaubt. Das Laden erfolgt mit `pickle.load()` (Theis, 2024, S. 286).

```
import pickle
with open("messdaten.pkl", "wb") as datei:
    pickle.dump(["10:00", 001, 3.7], datei) # Schreiben

with open("messdaten.pkl", "rb") as datei:
    daten = pickle.load(datei)           # Lesen
    print(daten)
```

Ein weit verbreitetes Format zum Austausch von Daten ist **JSON**, das die Daten als einfach lesbaren Text speichert, der auch mit anderen Programmiersprachen kompatibel ist. Das Einlesen erfolgt mit `json.load()` (Theis, 2024, S. 289).

```
import json
with open("konfiguration.json", "w") as datei:
    json.dump({"Messrate": 10, "Einheit": "Volt"}, datei)

with open("konfiguration.json", "r") as datei:
    konfiguration = json.load(datei)
    print(konfiguration)
```

4.6 Finalisierung und letzter Schliff

Gegen Abschluss des Programmierprojekts hin gibt es noch einige notwendige Schritte, um ein gutes Ergebnis zu erzielen. Es muss nicht nur der Code noch refaktoriert und in eine ausführbare Datei (Executable) überführt werden, sondern es gibt auch eine Reihe weiterer Optimierungspotenziale, die nachfolgend näher beleuchtet werden sollen:

4.6.1 Refactoring

Refactoring ist eine wesentliche Maßnahme zur Verbesserung von Code. Insbesondere die Verständlichkeit und Wartbarkeit werden dadurch erhöht. Bestehende Codeblöcke sollen optimiert werden, ohne deren Funktion zu verändern. Besonders die

Beseitigung von Redundanzen, die Umbenennung von Variablen in vielsagende Ausdrücke und Steigerung der Laufzeiteffizienz stehen dabei im Fokus (Knoll & Heedt, 2022, S. 517-523)

Ein zentrales Werkzeug des Refactorings ist die **Modularisierung**, worauf bereits im Kapitel 4.4.2 eingegangen wurde. Beim Refactoring sollen komplexe Codeblöcke in kleine, wiederverwendbare Funktionen umgewandelt werden, was die Wiederverwendung, das Testen und Debuggen einzelner Komponenten erleichtert (Theis, 2024, S. 74). Das nachfolgende, zwar recht simple, aber dennoch aufschlussreiche Beispiel verdeutlicht dies:

Ursprünglicher Code:

```
result1 = wert * 2 + 5
result2 = anderer_wert * 2 + 5
```

Nach dem Refactoring:

```
def berechne_ergebnis(wert):
    return wert * 2 + 5
```

Die Laufzeit kann wesentlich dadurch beeinflusst sein, welche **Algorithmen** gewählt werden. Ein weiteres Ziel des Refactorings ist es daher, ineffiziente Algorithmen durch leistungsfähigere Alternativen zu ersetzen (Downey, 2021, S. 251).

Ursprünglicher Code:

```
ergebnisse = []
for i in range(10):
    ergebnisse.append(i * i)
```

Nach dem Refactoring:

```
ergebnisse = [i * i for i in range(10)]
```

Schließlich ist auch die **Fehlerbehandlung** und **-vermeidung** ein wichtiger Teilaspekt der Refactorings. Alle Codepassagen, die bei neuerlicher Durchsicht als fehleranfällig eingeschätzt werden sollen durch gezielten Einsatz von try-except-Blöcken „entschärft“ werden, um Laufzeitfehler abzufangen. (Theis, 2024, S. 75). Eine strukturierte **Protokollierung** (logging) sollte implementiert werden, so dies nicht bereits im Entwicklungsstadium erfolgt ist, um die Fehleranalyse deutlich zu verbessern (Sweigart, 2020, S. 312). Eine fortgeschrittene Variante der Fehlerbehandlung sind automatisierte Unit-Tests, die dabei helfen, einzelne Codebestandteile zu überprüfen (Ulias, 2024, S. 108).

4.6.2 Dateien als Executable ausgeben

Die Erstellung einer Executable ermöglicht die Nutzung der GUI-Anwendung, ohne dass dafür Python auf dem Gerät installiert sein muss. Das wohl bewährteste Werkzeug dafür ist PyInstaller, das mit dem Befehl „*pip install pyinstaller*“ installiert wird.

Zur Umwandlung in eine ausführbare Datei muss das Terminal geöffnet und in das Verzeichnis gewechselt werden, in dem sich das Python-Skript befindet. Mit dem Befehl „*pyinstaller [Name der Datei].py*“ wird automatisch ein Verzeichnis *dist* erstellt, in dem sich die Executable befindet.

PyInstaller bietet dabei interessante Möglichkeiten zur Anpassung der ausgegebenen Datei:

- Mit dem Parameter *--onefile* wird lediglich eine einzige ausführbare Datei anstelle mehrerer Abhängigkeitsdateien erstellt
- *--noconsole* verhindert die Anzeige eines Konsolenfensters beim Start der GUI-Anwendung
- Mit *--icon=<icon.ico>* kann ein benutzerdefiniertes Icon für die ausführbare Datei gesetzt werden

(Theis, 2024, S. 458)

4.6.3 Desiderate

Im Rahmen der Bearbeitung des Projekts und des Studiums der Fachliteratur haben sich viele Desiderate ergeben, deren Aufnahme bzw. Behandlung nicht im Fokus der Projekterstellung liegt und den Rahmen der vorliegenden Arbeit sprengen würden. Es wäre wünschenswert gewesen, auch diese Aspekte näher zu beleuchten, weil sie im Zusammenhang mit dem Thema auch von Interesse wären. Dennoch sollen diese Themen kurze Erwähnung finden, da sie einen Impuls für weitere Projekte, die sich tiefer mit dem Thema GUI-Entwicklung – vielleicht auch in einem ingenieurswissenschaftlichen oder technischen Kontext – auseinandersetzen, bieten können.

4.6.3.1 Performance

Dass Python als Sprache für das Projekt gewählt wurde, weil es der raschen Prototypenentwicklung zuträglich ist, dafür aber nicht unbedingt zu den performantesten Sprachen zählt, wurde an früherer Stelle bereits erwähnt. Sollte stattdessen aber eher die Performance im Vordergrund stehen, wird man wohl eine andere Sprache wählen. Dennoch soll kurz auf das Thema der Performance(-steigerung) eingegangen werden, da einerseits Python durchaus Möglichkeiten zur Performance-Messung und -Steigerung bietet und die nachfolgenden Prinzipien andererseits allgemeingültig sind und für alle Programmiersprachen Anwendung finden.

Um die Performanz zu messen, muss zunächst der Ist-Zustand festgestellt werden, um treffsichere Verbesserungen vornehmen zu können. Besonders dann, wenn keine Daten zur Effizienz von Algorithmen vorliegen, ist es von Vorteil, konkrete Zahlen als Entscheidungsgrundlage für eine Überarbeitung des Codes zu haben (Knoll & Heedt, 2022, S. 533)

Für solche **Zeitmessungen** gibt es Module, die brauchbare Ergebnisse liefern. Das Python-Modul *time* beispielsweise erlaubt einfache Zeitmessungen, indem die Differenz zwischen zwei Zeitpunkten berechnet wird. Die Funktion *time.perf_counter()* liefert Messwerte in Sekunden als Fließkommazahl. Für hochfrequente oder sehr kurze Operationen kann die Messung mit *time* unzuverlässig sein. Das Modul *timeit* ist für hochpräzise Messungen ausgelegt. Die Funktion *timeit.timeit()*, führt eine Funktion mehrfach aus und mittelt die Laufzeit, was besonders hilfreich ist, um unterschiedliche Programmieransätze zu vergleichen und die effizienteste Variante zu wählen (Knoll & Heedt, 2022, S. 533-535).

4.6.3.2 Systematische Testmethoden

Qualitätssicherung ist ein Thema, das mit dem Anwachsen des Projektes zunehmend an Bedeutung gewinnt. Mithilfe von systematischen Testmethoden wie Assertions und Unittests können Fehler frühzeitig erkannt und eliminiert werden.

Mit **Assertions** können Annahmen darüber, wie sich Programmcode verhalten soll, gemacht werden. Sollte eine Bedingung nicht erfüllt sein, wird eine *AssertionError*-

Ausnahme ausgelöst. Dadurch können falsche Annahmen frühzeitig erkannt und Laufzeitfehler abgefangen werden (Knoll & Heedt, 2022, S. 558).

```
def add(a, b):  
    return a + b  
  
assert add(2, 3) == 5, "Test failed: add(2, 3) should be 5"
```

Unit-Tests prüfen einzelne Komponenten eines Programms unabhängig voneinander. Das Standardmodul *unittest* ermöglicht automatisierte Tests, um sicherzustellen, dass Funktionen so arbeiten wie es vom Entwickler vorgesehen ist (Knoll & Heedt, 2022, S. 560).

Dabei kommen je nach Anforderung unterschiedliche Testmethoden zum Einsatz:

- **Integrationstests** überprüfen die reibungslose Interaktion zwischen verschiedenen Teilen eines Programms und damit die Stabilität des Gesamtsystems.
- **Funktionstests**kontrollieren die Funktionalität der Anwendung in Bezug auf deren Anforderungen und Spezifikationen.
- **Performanztests**analysieren Reaktionszeiten, Skalierbarkeit und Ressourcennutzung in verschiedenen Anwendungsfällen.
- **Sicherheitstests**evaluieren potenzielle Schwachstellen der Software durch Penetration-Testing.
- **Usability-Tests geben ein Feedback zur** Benutzerfreundlichkeit, etwa durch systematische Rückmeldungen von Testnutzern.
- **Regressionstests**sollen sicherstellen, dass durch Änderungen am Code keine neuen Fehler entstehen (Ulias, 2024, S. 109–110).

Eine umfassende Teststrategie gewinnt insbesondere dann an Bedeutung, wenn der Code über einen längeren Zeitraum genutzt wird und mehrere Personen daran arbeiten. Auch wenn eine Integration der Anwendung in ein größeres Projekt vorgesehen ist oder von der Software wirtschaftliche oder sicherheitsrelevante Implikationen abgeleitet werden, ist ein systematisches und kontinuierliches Testregiment anzuraten (Knoll & Heedt, 2022, S. 565–566). Im Falle des vorliegenden

Projekts trifft dies jedoch nicht zu, da es über die experimentelle Phase der Prototypenentwicklung hinaus keine Anwendung findet.

4.6.3.3 Cross-Plattform-Kompatibilität in Python-GUI-Anwendungen

Die entwickelte Anwendung sollte im Idealfall unter Windows, Linux und macOS laufen und sich gleich verhalten. Auch wenn Tkinter mit den aufgelisteten Betriebssystemen kompatibel ist, gibt es Aspekte, die für uneingeschränkte Kompatibilität beachtet werden sollten (Moore A. D., 2021, S. 421).

Der häufigste Stolperstein dabei sind wohl die **Unterschiede bei den Dateisystemen**: Denn Windows verwendet ein laufwerksbasiertes System mit Backslashes (\) als Trennzeichen, wohingegen die Unix-basierten Betriebssysteme Linux und macOS ein hierarchisches Dateisystem und Forward Slashes (/) nutzen. So wäre ein hartcodierter Windows-Pfad wie „C:\User\Documents\file.txt“ unter Linux und macOS nicht gültig (Moore A. D., 2021, S. 423). In Python stehen zwei Module zur Verfügung, um mit Dateipfaden zu arbeiten:

Zwar bietet *os.path* einige Funktionen zur plattformunabhängigen Manipulation von Dateipfaden, während *pathlib* einen moderneren, objektorientierten Ansatz bietet, um bessere Lesbarkeit und Flexibilität zu gewährleisten (Moore A. D., 2021, S. 424-425)

Auch die **Groß- und Kleinschreibung von Dateinamen** wird unterschiedlich gehandhabt. Wo etwa Windows und macOS Dateinamen nicht case-sensitive verarbeiten, wird unter Linux und anderen Unix-Systemen ein Unterschied zwischen Groß- und Kleinschreibung gemacht. Deshalb sollte auf konsequent einheitliche Namenskonventionen für Dateien und Verzeichnisse geachtet werden. (Moore A. D., 2021, S. 426)

Unter Unix-Systemen sind **symbolische Links** (symlinks) weit verbreitet, auf Windows hingegen nicht. Darauf muss bei Pfadangaben geachtet werden, um unerwartetes Verhalten zu vermeiden. Mit *os.path.islink([Pfad])* kann geprüft werden, es sich um einen symbolischen Link handelt und *os.path.realpath([Pfad])* gibt den echten Pfad zurück (Moore A. D., 2021, S. 428).

Die unterschiedlichen Betriebssysteme verwenden außerdem unterschiedliche **Umgebungsvariablen**, um beispielsweise das Benutzerverzeichnis oder temporäre Ordner zu speichern. Unter Python können diese mit `os.getenv()` oder `os.path.expanduser()` abgerufen werden. Das Benutzerverzeichnis beispielsweise wird unter Windows mit %USERPROFILE%, unter macOS und Linux mit \$HOME adressiert (Moore A. D., 2021, S. 429).

Darüber hinaus bietet Tkinter zwar von sich aus eine hohe Kompatibilität zwischen den Betriebssystemen, einige kleinere Feinheiten müssen dennoch zusätzlich zu den bereits genannten beachtet werden:

Tkinter nutzt standardmäßig Systemschriften, welche je nach Betriebssystem unterschiedlich sind. Ein einheitliches Erscheinungsbild wird durch die Definition fester Schriftarten gewährleistet.

Durch unterschiedliche DPI-Skalierungen der einzelnen Betriebssysteme kann die Darstellung der Widgets leicht variieren, die Layout-Manager `grid()` und `pack()` sind aber für ein adaptiv-flexibles UI ausgelegt.

Die einzelnen Betriebssysteme verwenden teils andere Tastenkombinationen, mit `platform.system()` kann diese ausgelesen und entsprechend angepasst werden (Moore A. D., 2021, S. 421).

4.6.3.4 Verbesserung der GUI durch Themes

Eine grafische Benutzeroberfläche überzeugt nicht nur durch Funktionalität, auch das visuelle Design trägt wesentlich zur Benutzerfreundlichkeit und zur Annahme der Software durch den Endnutzer bei. Die Nutzererfahrung und Ästhetik können durch Einsatz von Farben, Schriftarten und Bildern verbessert werden. Mit dem `ttk`-Modul von Tkinter erhält man eine breitere Palette an Möglichkeiten zur optischen Gestaltung durch Styles, die für Widgets ein einheitliches Design vorgeben. Themes ermöglichen eine globale Änderung des Erscheinungsbildes in der gesamten Anwendung (Moore & Harwani, 2019, S. 299).


```
from tkinter import ttk    # import von ttk
style = ttk.Style()        # Style-Objekt erschaffen
style.theme_use("clam")    # Verwendung des Themes
```

Unter Umständen kann es sein, dass in eine Anwendung Logos oder eine spezifische Farbgebung integriert werden, um eine Corporate Identity zu wahren. Eine durchdachte Wahl der Gestaltungselemente ist jedenfalls ein wichtiges Element einer gelungenen GUI-Anwendung, was bei der Entwicklung stets mitzudenken ist.

4.6.3.5 Webkonnektivität in GUI-Anwendungen

Durch die Anbindung an das Internet kann die Funktionalität einer GUI-Anwendung deutlich erweitert werden. Externe Datenquellen können eingebunden, Webinhalte angezeigt, Updates, Dokumentationen oder Support bereitgestellt werden. Das *webbrowser*-Modul aus der Standardbibliothek von Python bietet die Möglichkeit, URLs im Standardbrowser des Benutzers zu öffnen. Mit der Funktion *webbrowser.open([url])* wird die angegebene URL geöffnet, mit einem zweiten (optionalen) Parameter kann die Art der Darstellung gesteuert werden: „o“ öffnet die URL in einem bestehenden Browserfenster, mit „1“ wird ein neues Browserfenster geöffnet, und „2“ öffnet einen neuen Tab in einem bestehenden Browserfenster (Ernesti & Kaiser, 2023, S. 1483).

```
import webbrowser
webbrowser.open("https://www.bulme.at/") # öffnet die URL
```

Wird außerdem der optionale Parameter *autoraise=True* verwendet, öffnet sich das neue Browserfenster im Vordergrund (Ernesti & Kaiser, 2023, S. 1484).

4.6.3.6 Multithreading in GUI-Anwendungen

Moderne Anwendungen beruhen darauf, dass mehrere Aufgaben parallel bearbeitet werden. Das betrifft auch – oder insbesondere – die GUI-Entwicklung, denn der Hauptthread sollte ausschließlich der Aktualisierung der grafischen Oberfläche und der Verarbeitung von Nutzereingaben gewidmet sein. Messwertaufnahmen, komplexe Berechnungen und andere Hintergrundprozesse werden in separaten Threads ausgeführt, damit die Anwendung „flüssig“ läuft. Mit dem Modul *threading* können Threads unter Python parallel ausgeführt werden, wodurch bestimmte Aufgaben unabhängig vom Hauptthread verarbeitet werden. Allerdings gilt das nur mit einer

Einschränkung, denn die Standard-Python-Implementierung CPython wird durch den Global Interpreter Lock (GIL) eingeschränkt. Der GIL verhindert, dass mehrere Threads gleichzeitig Python-Bytecode ausführen. Eine echte Parallelverarbeitung ist damit nur sehr begrenzt möglich. Somit ist *threading* zwar für asynchronen Benutzerinput oder Rechenoperationen im Hintergrund geeignet, CPU-intensive Prozesse profitieren jedoch nicht von paralleler Ausführung (Paz, 2018, S. 318).

Die Kommunikation zwischen Hintergrundprozessen und dem Hauptthread stellt häufig ein Problem der GUI-Entwicklung dar, beispielsweise dann, wenn der Fortschritt eines Prozesses nicht direkt im Hauptthread abgebildet werden kann, da dies zu sogenannten GUI-Freezes führen kann. Abhilfe schafft jedoch die Implementierung eines Fortschrittsbalkens (progress bar), um den Status eines Hintergrundprozesses anzuzeigen. Dabei übernimmt ein eigener Thread eine zeit- bzw. rechenintensive Aufgabe, während eine Callback-Funktion den Fortschrittsbalken im Hauptthread aktualisiert (Paz, 2018, S. 335).

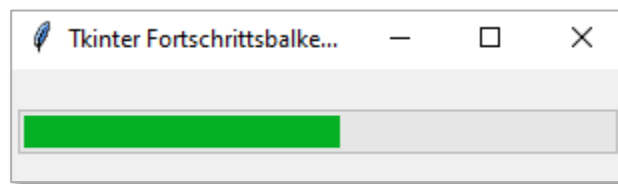


Abbildung 8 Ein Fortschrittsbalken zeigt den Status eines Hintergrundprozesses an

5 Dokumentation der praktischen Umsetzung: GUI-Projekt

Nachfolgend wird die praktische Umsetzung der in den vorangegangenen Kapiteln erarbeiteten theoretischen Grundlagen dokumentiert und erläutert. Dabei lässt sich der Workflow-Prozess grob in drei Phasen unterteilen: Planung, Umsetzung, Finalisierung.

5.1 „Von der Idee zum Konzept“: Planungs- und Konzeptionsphase

Die Planungsphase wurde mit einer Ist-Stands- und Projektziel-Analyse eingeleitet, dabei sollte festgestellt werden, auf welche Ressourcen bereits zurückgegriffen werden

konnte, auf welches Ergebnis im Zuge des Projekts hingearbeitet werden sollte und mithilfe welcher Schritte dies zu erreichen sei.

Als **Ziel des Projekts** wurde die Erarbeitung einer Anwendung mit grafischer Benutzeroberfläche festgelegt. Die Applikation sollte über den CAN-Bus mit einem externen Messaufbau bidirektional kommunizieren. Dabei sollen Konfigurationen nach einem festgelegten Muster erstellt, gespeichert und an die Peripherie übermittelt sowie Messdaten empfangen und für den Endnutzer grafisch aufbereitet ausgegeben werden.

Der CAN-Bus wurde als Übertragungsmedium gewählt, da er sich durch seine robuste, fehlerresistente Kommunikation auszeichnet und in industriellen Anwendungen weit verbreitet ist, wodurch das Controller Area Network mit seiner einfachen Erweiterbarkeit und hohen Effizienz auch weiterhin einen großen Stellenwert einnimmt. Überdies sollte es eine Ambition sein, mit einem fortgeschrittenerem Kommunikationsmedium zu arbeiten als beispielsweise I2C oder SPI.

Bereits **vorhandene Wissens- und Kompetenzressourcen** in diesem Bereich erleichterten den Einstieg in das Projektthema zudem deutlich. Auch wurde recht rasch eine passende API für die Schnittstellenkommunikation identifiziert. Mit dem PCAN-USB-Interface, entwickelt von der PEAK-System Technik GmbH aus Darmstadt, war eine einfache Anbindung an CAN-Netzwerke möglich. Dankenswerterweise wurde von dieser Firma ein Modell für die Projektzwecke zur Verfügung gestellt. Das Paket enthält die Kommunikationssoftware PCAN-View und die Programmierschnittstelle PCAN-Basic, was die Entwicklung einer GUI-Anwendung, die an das Controller Area Network angebunden sein sollte, ebenso deutlich unterstützte wie die zur Verfügung gestellten Gerätetreiber.

In weitere Folge waren Entscheidungen zu klären, welche Programmiersprache verwendet werden sollte. Von den Anforderungen und den Rahmenbedingungen ausgehend fiel die Entscheidung auf Python, die Gründe hierfür sind ausführlich unter Punkt 4.1 beschrieben. Als passendes GUI-Framework wurde Tkinter gewählt, da es in Kombination mit Python einen dynamischen, iterativen Entwicklungsprozess erlaubt, alle weiteren Gründe für die Nutzung von Tkinter sind in den Kapiteln 4.2.1 und 4.3.1 aufgelistet.

Nachdem diese Entscheidungen getroffen waren, war der Weg für konkrete Überlegungen, welche **Funktionen die GUI-Anwendung** konkret erfüllen soll, geebnet. Neben der Konfiguration des Messaufbaus und dessen Kommunikation über den CAN-Bus galt es, eine übersichtliche und benutzerfreundliche Oberfläche zu gestalten, bei der die einfache Bedienbarkeit und die User Experience im Mittelpunkt stehen. Dafür standen – neben anderen – vor allem die nachfolgenden Überlegungen im Mittelpunkt:

- Welche Anforderungen stellt der Endnutzer an die Anwendung?
- Welche Widgets würden den Anforderungen an die Applikation am ehesten entsprechen?
- Mit welchen Nutzerinteraktionen sollte die Anwendung umgehen können?
- In weiterer Folge: Wie sollten die Widgets angeordnet werden und welcher Layout-Manager würde sich für diese Zwecke am ehesten eignen (bezüglich Layout-Manager sei auf die Ausführungen unter Punkt 4.3.1.1 verwiesen)

Ergebnis dieses Kalküls war, dass neben Frames, welche unverzichtbare Elemente zur Einteilung des Anwendungsfensters sind, auch Labels, Buttons, Entry, Comboboxes, Checkbuttons, Listboxes und ScrolledText notwendig sind, um den Anforderungen bei der Endnutzung zu entsprechen. Labels sind universelle Textelemente, die unter anderem zur Beschriftung anderer Widgets dienen, weshalb ihre Verwendung außer Diskussion stand. Buttons dienen der einfachen und unmittelbaren Nutzerinteraktion, etwa zum Un-/Initialisieren der Kommunikation über den CAN-Bus. Entryfelder dienen der frei wählbaren Eingabe durch den Endnutzer, etwa bei der Eingabe eines Delays zwischen einzelnen Messschritten. Hier weist ein Entry den Vorteil auf, einen individuellen, nicht vorgegebenen Wert vom Nutzer entgegenzunehmen. Die Combobox bietet als „Drop-down“ eine Auswahl vorgegebener Werte und eignet sich damit im Besonderen für die Auswahl der (fest vorgeschriebenen) Baudraten und die Auswahl der Einheit für das Delay – Milli- oder Mikrosekunden. Checkbuttons nehmen binäre Werte vom Benutzer entgegen und eignen sich daher besonders zur

Konfiguration der Messaufbauten. Mit einem Klick kann so beispielsweise ein ADC(-Channel), MOSFET, GPIO-Pin oder Komparator in einen Messaufbau eingebunden oder davon entbunden werden. Die Listbox ist für die Konfiguration des Messablaufs geeignet, der ScrolledText für die Ausgabe der Messergebnisse und das Logging. Ergänzt werden sollte das Design mit einer Menüleiste, Kontextmenüs und Dialogfeldern, um die User Experience weiter zu verbessern.

Nachdem diese grundlegenden Überlegungen abgeschlossen waren, konnten erste Skizzen der Benutzeroberfläche erstellt werden, um die optimale Anordnung der Widgets auszutesten und Rückmeldungen dazu einzuholen. Ein früher Entwurf ist nebenstehend abgebildet.

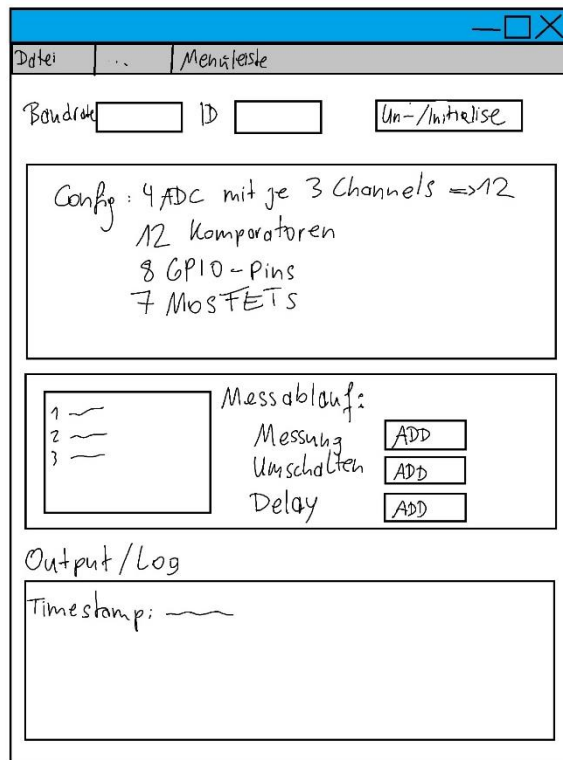


Abbildung 9 Früher Entwurf des GUI-Designs in der Planungsphase

Abgerundet wurde die Planungs- und Konzeptionsphase durch die Erstellung eines detaillierten Zeitplans mit definierten Meilensteinen. Dabei wurde festgelegt, welche Errungenschaften bis zu welchem Zeitpunkt erreicht sein sollten. Dabei wurden sowohl potenzielle Stolpersteine – wie Kompatibilitätsprobleme und die Implementierung der Fehlerbehandlung bei der CAN-Kommunikation – identifiziert als auch erste Strategien zur Risikominimierung festgelegt, als auch ein zeitlicher Rahmen für die weitere Vorgehensweise festgelegt.

Zeitpunkt	Meilenstein	Erklärung/Ziel
Ende der Herbstferien	Feststellen der Anforderungen und Planung	Anforderungen der Software, Protokolle, Funktionen des GUI feststellen

Ende der Weihnachten	CAN-Bus Kommunikation	Kommunikation zwischen Software und CAN-Bus, Sensorwerte auslesen
Ende der Semester- ferien	Implementierung der Kommunikation in der GUI + Design	Darstellung der Sensorwerte
Bis Ende März	Testen und Debugging	Stabil laufende Anwendung, die fehlerfrei Sensorwerte über den CAN- Bus ausliest und anzeigt.
Bis Ende April	Schriftliche Dokumentation	Zusammenfassung: Dokumentation und Projektergebnisse

Tabelle 10 Selbst gesetzte Meilensteine bei der Erarbeitung des Projekts

Als letzten Schritt der Konzeptionsphase wurden die Literatursuche und das Literaturstudium eingeleitet. Dafür wurden zunächst unterschiedliche Bibliothekskataloge sowie Online-Quellen nach passender Fachliteratur zu den in der vorliegenden Arbeit behandelten Themen (CAN, Python, GUI-Entwicklung etc.) durchsucht und die entsprechenden Publikationen entlehnt, erworben und strukturiert durchgesehen, um für die Arbeit relevante Passagen zu exzerpieren. Die Arbeit mit der Fachliteratur begann in der Konzeptionsphase, aber hat das Entstehen der vorliegenden Arbeit bis zum finalen Text hin begleitet.

Die Recherche zu grafischen Benutzeroberflächen begann mit der Lektüre von leicht zugänglichen (und mindestens gleich verbindlichen wie gedruckten) Quellen im Internet:

- <https://docs.python.org/3/library/tkinter.html>
- <https://tkdocs.com/tutorial/onepage.html>

In der nächsten Projektphase begann mit der schrittweisen Entwicklung des Prototyps:

5.2 Umsetzungsphase: Implementierung und Herausforderungen

Der Entwurf eines Kommunikationsprotokolls bildet quasi die Schnittstelle zwischen der Konzeptions- und Umsetzungsphase des Projekts. Das Ziel des Kommunikationsprotokolls war es, sowohl die GUI-seitig erstellten Konfigurationen zum Messaufbau hin, als auch die Messwerte über den CAN-Bus in umgekehrter Richtung zu übertragen. Dafür werden verschiedene Parameter wie ADC-Kanäle, Komparatoren und Schaltlogiken übertragen. Die Kommunikation erfolgt über definierte Nachrichten („Frames“), die aus einer CAN-ID, einem Index, einem Sub-Index sowie Datenbytes bestehen.

CAN-ID	Index	Sub-Index	Datenbyte 1	Datenbyte 2	Datenbyte 3	Datenbyte 4	Datenbyte 5	Datenbyte 6
--------	-------	-----------	-------------	-------------	-------------	-------------	-------------	-------------

Tabelle 11 Grundsätzlicher CAN-Frame-Aufbau

Im Rahmen des Projekts wurde angenommen, dass es außer dem CAN-USB-Converter und dem Messaufbau keinerlei andere Bus-Teilnehmer gibt, was die Adressierung wesentlich vereinfacht.

Bei der erstmaligen Konfiguration muss die Baudrate, die dafür als Hex-Wert in das erste Datenbyte geschrieben wird, einmalig festgelegt werden. Werden die

125 kbit/s	0x01
250 kbit/s	0x02
500 kbit/s	0x04
800 kbit/s	0x08
1 Mbit/s	0x10

Einstellungen vom Messaufbau übernommen, so erfolgt eine Antwort mit dem Sub-Index „0x01“ – wenn nicht, mit dem Sub-Index „0x08“

Tabelle 12 Baudraten

Nach der erstmaligen Konfiguration sehen die Standard-Frames für die Datenübertragung wie folgt aus, der Subcode gibt dabei an, welche Nutzdaten übermittelt werden:

Konfiguration Messaufbau

CAN-ID	Index	Sub-Index	Datenbyte 1	Datenbyte 2	Datenbyte 3	Datenbyte 4	Datenbyte 5	Datenbyte 6
CAN-Offset + 0x001	Code	Subcode	Messaufbau Nr.	12 ADC-Channels		12 Komparatoren		-

Änderung Messaufbau

CAN-ID	Index	Sub-Index	Datenbyte 1	Datenbyte 2	Datenbyte 3	Datenbyte 4	Datenbyte 5	Datenbyte 6
CAN-Offset + 0x001	Code	Subcode	Messaufbau Nr.	8 GPIO-Pins	7 MOSFETs			

Delay

CAN-ID	Index	Sub-Index	Datenbyte 1	Datenbyte 2	Datenbyte 3	Datenbyte 4	Datenbyte 5	Datenbyte 6
CAN-Offset + 0x001	Code	Subcode	Messaufbau Nr.	Delay in μ s				

Tabelle 13 Datenframes bei der Kommunikation über den CAN-Bus

Sowohl die CAN-ID wie auch die Baudrate sollen individuell gewählt werden können, in weiterer Folge werden die strukturierte Abfolge von Messprozessen und Delays übertragen sowie Fehler erkannt und zurückgemeldet.

Der gesamte Messprozess setzt sich aus drei Phasen zusammen:

1. die GPIOs und Mosfet-Relais werden entsprechend der Konfiguration geschaltet
2. die ADCs und Komparatoren erfassen die Messwerte
3. Verzögerung

Diese Phasen lassen sich in beliebiger Reihenfolge und Häufigkeit kombinieren, sodass spezifische Messprozesse flexibel gestaltet werden können. Die erfassten Werte werden anschließend über den CAN-Bus übertragen.

Gemäß diesen Anforderungen und Grundlage des frühen Entwurfs (s. Abbildung 9 Früher Entwurf des GUI-Designs in der PlanungsphaseAbbildung 9) wurde ein grafisches Interface entworfen. Auf die Entscheidungen und Parameter bezüglich Layout-Manager und Widgets wurde bereits in den Kapiteln 4.3.1.1 und 4.3.1.2 eingegangen. Dabei hat sich grid() als die passende Wahl für den Layout-Manager herauskristallisiert. Das dabei verfolgte Konzept eines Rasters, in dem Widgets in Zeilen und Spalten angeordnet werden, fördert die Entwicklung von übersichtlich strukturierten Oberflächen.

Die Layout-Manager arbeiten mit einer Hierarchie von übergeordneten Master- und untergeordneten Slave-Widgets. Grid() sorgt je nach übergebenen Parametern für die passende Anordnung der Slave-Widgets im Master-Widget. Die wichtigsten Parameter

sind wohl *column* und *row*, mit denen – ähnlich wie bei einem Schachbrett – die Position im Raster ausgewählt wird. Mit *columnconfigure* und *rowconfigure* und dem Parameter *weight* wird der zur Verfügung gestellte Platz innerhalb des Parent-Frames festgelegt. In Abbildung 5 wird anschaulich, wie die Positionierung der einzelnen Widgets in einem Raster erfolgt.

Exemplarisch soll die Anwendung dieses Layout-Managers (und seine Praktikabilität) anhand jenes Teils der Nutzeroberfläche für die Konfiguration des Messaufbaus – also der Auswahl der zu verwendenden Channels der Analog-Digital-Wandler und der Komparatoren – dargestellt werden.

```
# Hinzufügen eines Frames für die Konfiguration, Parent ist root, das Padding
ist global definiert
measurement_frame = ttk.Frame(root, padding = GLOBAL_PAD_FRAME)
# Das Frame wird in der ersten Spalte und der dritten Zeile des root-Frames
platziert
measurement_frame.grid(column=0, row=2, sticky=(N, W, E, S))

# Platzieren eines Labels innerhalb des measurement_frame
choose_config_label = ttk.Label(measurement_frame, text="Choose
Configuration", font= font_title)
# Position: Erste Spalte, erste Zeile
choose_config_label.grid(column=0, row=0, padx=10, pady=GLOBAL_PAD_Y,
sticky=W)

# Platzieren eines Frames innerhalb des measurement_frame
adc_frame = ttk.Frame(measurement_frame, padding=GLOBAL_PAD_FRAME,
relief="groove", borderwidth=2)
# Position: Erste Spalte, zweite Zeile

adc_frame.grid(column=0, row=1, sticky=(N, W, E, S))
# Platzieren eines weiteren Frames innerhalb des measurement_frame
comp_frame = ttk.Frame(measurement_frame, padding=GLOBAL_PAD_FRAME,
relief="groove", borderwidth=2)
# Position: Zweite Spalte, zweite Zeile
comp_frame.grid(column=1, row=1, sticky=(N, W, E, S))
```

Choose Configuration									
ADC 1	ADC 2	ADC 3	ADC 4		Comparators				
☐ Channel 1	☐ Channel 1	☐ Channel 1	☐ Channel 1	Fill all	☐ Comp. 1	☐ Comp. 4	☐ Comp. 7	☐ Comp. 10	Fill all
☐ Channel 2	☐ Channel 2	☐ Channel 2	☐ Channel 2	Clear all	☐ Comp. 2	☐ Comp. 5	☐ Comp. 8	☐ Comp. 11	Clear all
☐ Channel 3	☐ Channel 3	☐ Channel 3	☐ Channel 3		☐ Comp. 3	☐ Comp. 6	☐ Comp. 9	☐ Comp. 12	

Abbildung 10 Die Eingabemaske zur Konfiguration des Messaufbaus

Der rot umrahmte Teil zeigt das **Frame für die Konfiguration des Messaufbaus**, das selbst innerhalb vom Hauptfenster *root* angeordnet ist, jedoch selbst **zwei untergeordnete Frames** (grün umrahmt) enthält. Innerhalb der Slave-Frames zeigt sich die charakteristische **Rasteranordnung der Elemente** durch *grid()*.

Nachdem die grundsätzliche Struktur durch Frames und den Layout-Manager geschaffen ist, werden im nächsten Schritt die Widgets implementiert. Diese können auf zweierlei Arten erstellt und positioniert werden – in einer Variablen gespeichert werden oder nicht. Obwohl die Oberfläche etwas über 100 Widgets enthält, fiel die Entscheidung auf die Speicherung innerhalb einer Variable, da es zwar aufwändiger, aber jedoch übersichtlicher ist. Außerdem soll die Möglichkeit bestehen nachträglich auf das Widget zugegriffen werden – vor allem während der iterativen Prototypenentwicklung:

Speicherung in einer Variablen	<pre>check_all_button = ttk.Button(adc_frame, text="Fill all", command=lambda: check_all(channel_vars)) check_all_button.grid(column=5, row=1, padx=10, pady=5, sticky=E)</pre>
Unmittelbare Erstellung und Positionierung	<pre>ttk.Button(adc_frame, text="Fill all", command=lambda: check_all(channel_vars)).grid(column=5, row=1, padx=10, pady=5, sticky=E)</pre>

Tabelle 14 Arten der Widget-Erstellung

Davon abgesehen sind beide Varianten gleichwertig, da die Garbage Collection die Widgets auch dann nicht entfernt, wenn keine Variable darauf verweist.

Nach diesem Schema wurden alle benötigten Widgets in das *grid()*-Schema eingefügt und mit entsprechenden Parametern versehen, die Hierarchie der Widgets sieht dabei wie folgt aus:

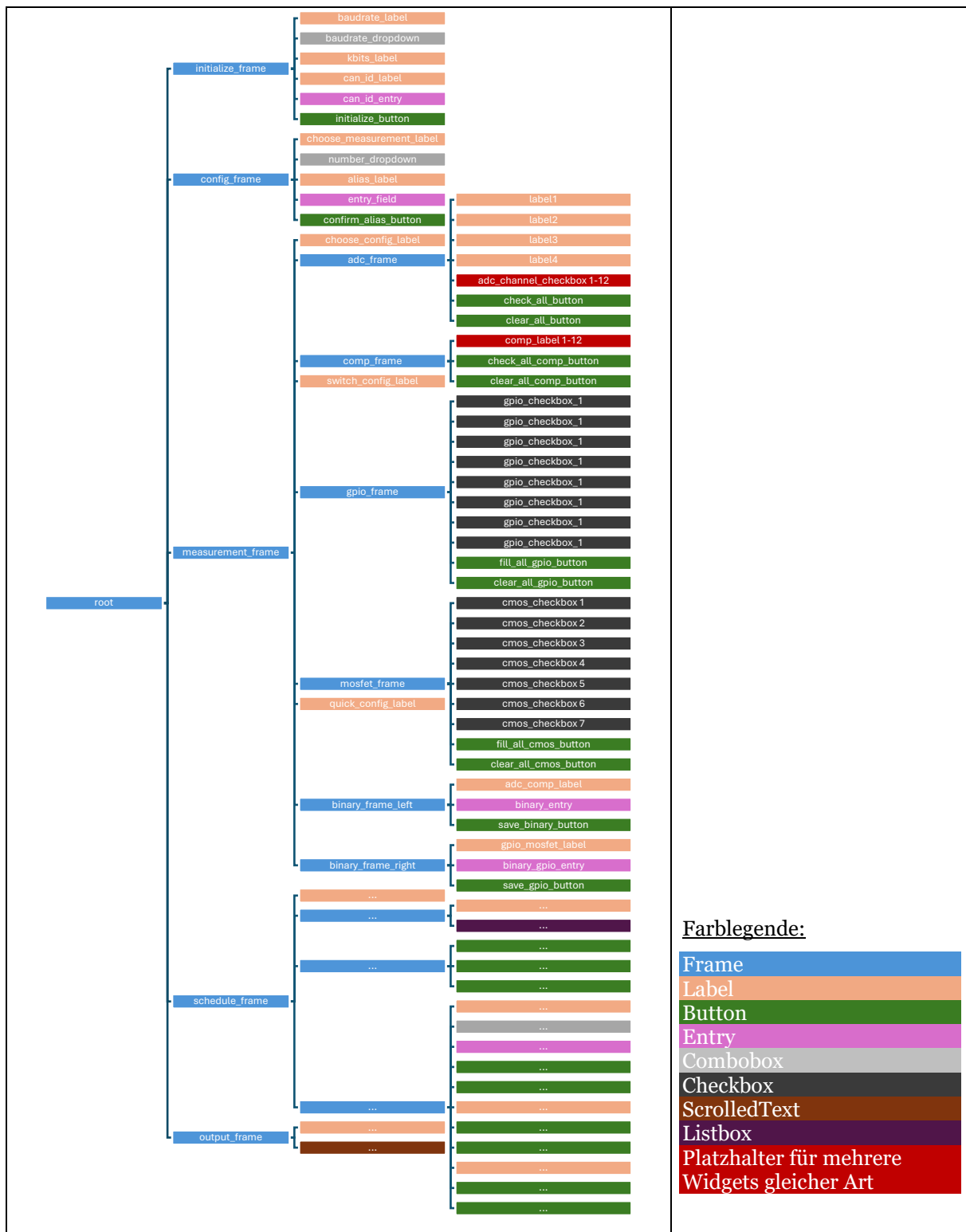


Abbildung 11 Die Darstellung der Widget-Hierarchie

Ereignisbehandlung durch Callbacks

Während dieser „Design-Phase“ wurden Callbacks für Benutzerinteraktionen ausschließlich *Dummy-Funktionen* mit der Anweisung *pass* als implementiert. Die Gestaltung der Oberfläche und Anordnung der Elemente hatte Vorrang. Nachdem für

die grafische Umsetzung ein erster Prototyp gefertigt war, sollten die Platzhalterfunktionen mit Leben erfüllt werden, um die Anwendung interaktiv zu machen. Bei Tkinter läuft ein *mainloop*, der Benutzereingaben wie Tastenanschläge, Mausbewegungen und -klicks vom Betriebssystem entgegennimmt.

```
root.mainloop()
```

Das Framework ermittelt automatisch, auf welches Widget sich ein Ereignis bezieht – ob ein Button geklickt wird oder ob bzw. welches Texteingabefeld aktiv ist, während ein Tastenanschlag erfolgt. Bei dem Erstellen der meisten Widgets kann eine Callback-Funktion für Nutzerinteraktionen festgelegt werden. Das nachfolgende Beispiel verdeutlicht, wie ein Klick auf einen Button eine Funktion aufruft, die alle booleschen Variablen eines Containers auf *True* setzt. Ein Klick auf einen anderen Button hingegen ruft eine Funktion auf, die alle Variablen auf *False* setzt.

```
def check_all(vars):
    for var in vars:
        var.set(True)

def uncheck_all(vars):
    for var in vars:
        var.set(False)
```

```
check_all_button = ttk.Button(adc_frame, text="Fill all", command=lambda:
check_all(channel_vars))

clear_all_button = ttk.Button(adc_frame, text="Clear all", command=lambda:
uncheck_all(channel_vars))
```

Damit der Callback-Funktion auch Parameter übergeben werden können, muss sie als Lambda-Funktion ausgeführt sein, andernfalls reicht der Name der Funktion zur Ausführung:

```
save_binary_button = ttk.Button(binary_frame_left, text="Apply Quickconfig",
command=save_binary_to_config)
```

Verknüpfungen mit `bind()`, Events und virtuelle Events

Für alle Benutzereingaben, die nicht mit einer Widget-spezifischen Callback-Funktion – wie eben das Klicken eines Buttons – behandelt werden, besteht die Notwendigkeit

bzw. Möglichkeit, diese Eingaben abzufangen und über *bind()* mit einer Funktion zu verknüpfen.

Ein ganz simples Beispiel dafür ist die Verknüpfung der *Escape-Taste* mit der Funktion *quit()*, die das Programm beendet:

```
def exit_program(event=None):
    root.quit()
    root.destroy()

root.bind('<Escape>', lambda event: exit_program())
```

Tkinter bietet eine Vielzahl von verknüpfbaren Ereignissen, wie beispielsweise bestimmte Tastenanschläge oder Mausbewegungen. Konkrete Verwendung im Projekt findet jedoch nur *<FocusIn>*. Dieses Ereignis wird getriggert, wenn das damit verbundene Widget über einen Mausklick (oder die Tabulatortaste) den Fokus erhält.

```
can_id_entry.bind("<FocusIn>", clear_can_id_placeholder())
```

Neben den grundlegenden Events gibt es in Tkinter auch *virtuelle Ereignisse*, die durch doppelte spitze Klammern gekennzeichnet sind. Häufig verwendete virtuelle Ereignisse sind beispielsweise *<<Cut>>*, *<<Copy>>* und *<<Paste>>* zum Ausschneiden, Kopieren und Einfügen von Inhalten in bzw. aus der Zwischenablage.

Im Projekt finden die Events *<<ComboboxSelect>>* und *<<ListboxSelect>>* Verwendung. Sie werden ausgelöst, sobald sich die Auswahl einer Combobox oder Listbox ändert. Im nachfolgenden Beispiel sollen unterschiedliche Buttons für den Benutzer aktiv oder inaktiv geschaltet werden, je nachdem welches Element einer Listbox aktiv ist.

```
def update_edit_button_state(event):
    selected_index = listbox.curselection()
    if selected_index and "Switching:" in listbox.get(selected_index):
        edit_gpio_cmos_entry_button.state(['!disabled'])
    else:
        edit_gpio_cmos_entry_button.state(['disabled'])
    if selected_index and "Measurement:" in listbox.get(selected_index):
        edit_measurement_entry_button.state(['!disabled'])
    else:
        edit_measurement_entry_button.state(['disabled'])
    if selected_index and "Delay:" in listbox.get(selected_index):
        apply_button.state(['!disabled'])
    else:
        apply_button.state(['disabled'])
```

```
listbox.bind('<<ListboxSelect>>', update_edit_button_state)
```

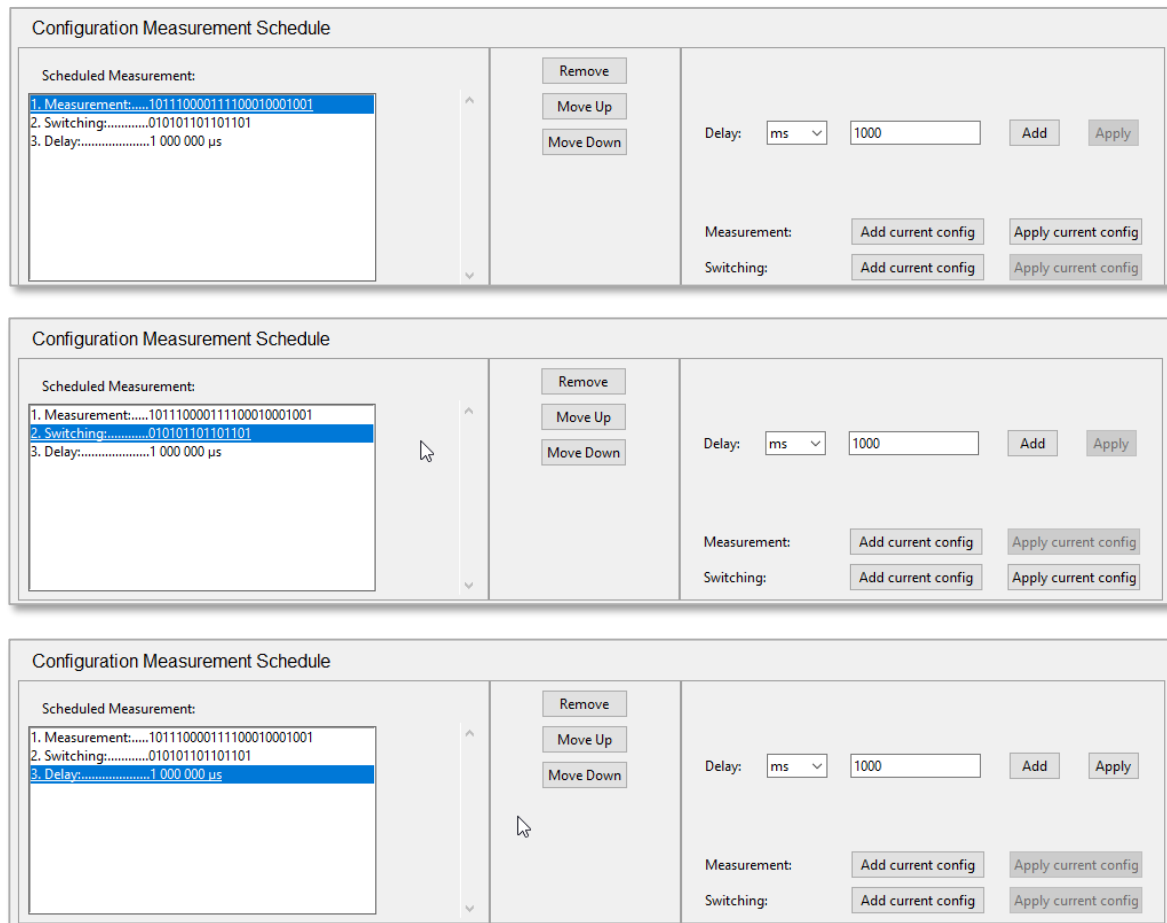


Abbildung 12 Mit dem <<ListboxSelect>>-Event wird eine Funktion zur Zustandsänderung von Buttons aufgerufen

Eingabefelder und deren Inhalt

Checkbuttons, *Radiobuttons*, *Listboxes*, *Comboboxes*, *ScrolledTexts* und *Entry-Fields* können Werte über eine verknüpfte Variable speichern, die mit dem *textvariable*-Parameter definiert wird. Diese Variablen müssen angelegt und verwaltet werden. Ausgelesen wird der Wert eines dieser Widgets mit *get()*. Mit *delete()* und *insert()* kann der Inhalt (partiell) gelöscht oder beschrieben werden.

Ein Beispiel für diese Methoden ist:

```
# Speichert den Inhalt des Entryfields in der Variablen „delay“
delay = delay_var.get()
# Löscht den Eintrag mit dem Index „selected_index“ aus der Listbox
listbox.delete(selected_index)
```

```
# fügt einen f-String am Index „selected_index“ in die Listbox ein
listbox.insert(selected_index, f"{listbox.size() + 1}.
Measurement:".ljust(20, '.') + f"{new_binary_string}")
```

Integration der CAN-Kommunikation

Parallel zur GUI-Entwicklung wurde die Schnittstelle zur Kommunikation über den CAN-Bus über die PCANBasic-API implementiert. Dabei wurden zunächst Basisfunktionen wie das Senden und Empfangen von CAN-Nachrichten realisiert. Die Anbindung an den Messaufbau erwies sich als Herausforderung, insbesondere im Hinblick auf das asynchrone Verhalten der Nachrichten und das Handling von Ausnahmen.

```
def WriteFrame(can_message):
    channel = PCANBasic.PCAN_USBBUS1
    message = TPCANMsg()          # TPCANMsg Objekt erstellen
    message.ID = 0x001             # CAN-ID
    message.MSGTYPE = PCANBasic.PCAN_MESSAGE_STANDARD # Standardframes
    message.LEN = 8                # Nachrichtenlänge in Byte
    message.DATA = can_message

    try:
        res = pcan.CAN_Write(channel, byref(message))
        status = TPCANStatus(res)
        print(f"Message sent with status: {status}")
    except Exception as e:
        print(f"Exception occurred: {e}")
```

Bedauerlicherweise handelt es sich hierbei lediglich um theoretische Überlegungen. Ein Funktionalitätstest war mangels Vorhandenseins eines Messaufbaus leider nicht möglich.

Messageboxes, Filedialogs, Speicherung von Messdaten

In der Entwicklung der Anwendung wurden zwei Arten von Dialogfenstern verwendet: **Messageboxes** und **Filedialogs**. Erstere dienen zum Abbilden von Warn-, Fehlermeldungen und Bestätigungen. Filedialogs werden zum Speichern von Konfigurations- und Messdateien und zum Laden von früheren Konfigurationen

verwendet. So wird beispielsweise eine Warnung ausgegeben, wenn keine *config.txt*-Datei im Root-Ordner gefunden wird.

```
config_file_path = os.path.join(os.path.dirname(__file__), "config.txt")
if not os.path.exists(config_file_path):
    messagebox.showwarning("Warning", "Config file not found.")
```

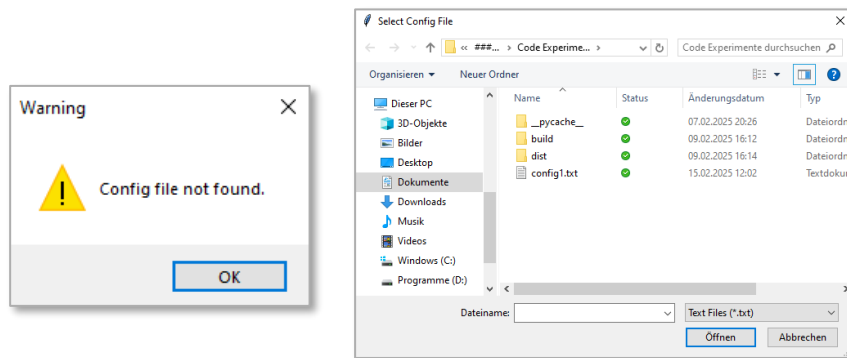


Abbildung 13 MessageBox und Filedialog in der Anwendung

Die Anwendung benötigt eine zuverlässige Speicherung von Mess- und Konfigurationsdaten. Die unterschiedlichen Formate dafür sowie ihre Vor- und Nachteile wurden bereits unter Punkt 4.5.2 erörtert. Aufgrund der einfachen und gut lesbaren Speicherung wurde das schlichte TXT-Format gewählt. Dabei sollten mögliche Fehler abgefangen werden, etwa wenn eine Datei nicht existiert. Ein Beispiel aus der Praxis:

```
def open_config():
    config_file_path = filedialog.askopenfilename(title="Select Config File",
filetypes=[("Text Files", "*.txt")])
    if config_file_path:
        with open(config_file_path, "r") as f:
            content = f.read()
            messagebox.showinfo("Config File", f"Config file content:\n{content}")
    else:
        messagebox.showwarning("Warning", "No config file selected.")
```

Während dieser Phase, die wohl mit Abstand am meisten Zeit in Anspruch nahm, entwickelte sich ein tiefes Verständnis für die GUI-Entwicklung beim Autor dieser Arbeit. Es darf allerdings nicht unerwähnt bleiben, wie wichtig Nutzerfeedback

während der Erstellung einer solchen Anwendung ist. Wiederholtes Testen und Anpassen führte zu einer stetigen Verbesserung der Applikation.

Die iterative und durch regelmäßiges Feedback angeleitete Weiterentwicklung führt zur dritten und letzten Stufe des Projekts:

5.3 Finalisierungsphase – letzter Feinschliff und Abschluss

In der letzten Phase des Projekts lag der Fokus auf der **Optimierung** des Programmcodes.

Dafür wurden beispielsweise Funktionen und Objekte in Nebendateien ausgelagert, um einen besseren Überblick über den Code zu bekommen. Eine weitere wichtige Maßnahme im Sinne der **Modularisierung** war die konsequente Anwendung objektorientierter Programmierkonzepte sowie eine strikte Trennung zwischen Logik, Benutzeroberfläche und Datenverarbeitung im Code.

Der Code sollte nicht nur den Richtlinien guter Programmierung, die an früherer Stelle besprochen wurden, entsprechend gut les- und wartbar sein, auch die Stabilität und Performanz der Anwendung sollte verbessert werden. Daher kamen in dieser Projektphase Strategien der systematischen **Refaktorisierung** zur Anwendung: Unnötige Codewiederholungen wurden möglichst eliminiert und vulnerable Codestellen von try-except-Blöcken umgeben, damit Laufzeitfehler nicht zu Programmabstürzen führen. Insbesondere unvorhergesehene Benutzerinteraktionen oder Kommunikationsprobleme mit dem CAN-Bus könnten zu Fehlern führen, die während der Entwicklungsphase nur schwer antizipiert werden können. Nachfolgendes Beispiel soll den gezielten Einsatz von *try-except*-Blöcken bei Benutzereingaben im Code veranschaulichen, um mögliche Fehler vorwegzunehmen:

```
def validate_input():
    try:
        messaufbau = int(entry.get())
        if messaufbau < 0 or messaufbau > 255:
            raise ValueError("Der Messaufbau muss zwischen 0 und 255 liegen.")
        messagebox.showinfo("Erfolg", "Eingabe erfolgreich!")
    except ValueError as e:
        messagebox.showerror("Fehler", f"Fehler: {e}")
```

Eine andere, doch ebenso zielführende Herangehensweise, um Laufzeitfehler zu verhindern, ist die **Eingabeverifizierung** bei Entry-Feldern und Buttons. Damit können die Eingaben des Endnutzers gesteuert werden und nur valide Eingaben zugelassen werden. Dabei stellt TKinter drei Methoden der Validierung zur Verfügung:

- *validate*: Definiert den Moment, wann die Validierung stattfindet. Typischer Anwendungsfall ist *validate= „key“*, wodurch die Funktion zur Prüfung mit jedem Tastenanschlag aufgerufen wird.
- *validatecommand* gibt die Funktion an, mit der die Nutzereingabe überprüft wird.
- *invalidcommand* wird bei ungültigen Ausgaben ausgeführt.

Ein Beispiel aus dem Programmcode soll hierfür herausgegriffen und Schritt für Schritt erklärt werden. Zweck des nachfolgenden Codes ist es, die Eingabe in ein Entry-Feld durch den Benutzer so zu beschränken, dass das Feld nur mit einem 24-stelligen binären Wert gefüllt werden kann. Jede Stelle bildet einen Channel eines ADCs ab, der entweder eine Messung durchführen soll:

Zunächst wird eine StringVar initialisiert, die als Platzhaltertext in dem Entryfield fungiert. Gleich darauf folgt das Entry-Feld *binary_entry*, das über den Textvariable-Parameter mit *binary_var* verknüpft und mit *grid()* im Fenster positioniert wird.

```
binary_var = StringVar(value='000 000 000 000 000 000 000 000') #  
Initialisiert die Variable binary_var  
# Erstellt ein Entry-Widget, das die Variable binary_var nutzt und die  
Funktion validate_and_format_binary  
binary_entry = ttk.Entry(binary_frame_left, textvariable=binary_var,  
validate='key', validatecommand=vcmd, width=30, justify='right')  
binary_entry.grid(column=1, row=0, padx=10, pady=GLOBAL_PAD_Y, sticky=W)  
# Bindet die Funktion clear_placeholder an das Event <FocusIn>  
binary_entry.bind("<FocusIn>", clear_placeholder)
```

Sobald das Event *<FocusIn>* detektiert wird, also ein Mausklick auf den Platzhaltertext erfolgt, wird die *clear_placeholder*-Funktion aufgerufen, die (nur) den Platzhalter löscht, bei einem nochmaligen Aktivieren des Felds – etwa um Eingaben des Nutzers zu korrigieren – jedoch nicht:

```
def clear_placeholder(event):  
    # Löscht den Platzhaltertext, wenn das Entry-Widget angeklickt wird  
    if binary_var.get() == '000 000 000 000 000 000 000 000':  
        binary_var.set('')
```

Bei der Eingabe von Ziffern in das Feld wird das *validatecommand* *vcmd* aufgerufen, das die Funktion *validate_and_format_binary* im root Fenster an die register-Methode übergibt. Das '%P' ist ein Formatierungszeichen, das in Tkinter verwendet wird und den Wert des Eingabefeldes repräsentiert.

```
vcmd = (root.register(validate_and_format_binary), '%P')
```

Als nächstes wird die Funktion *validate_and_format_binary* aufgerufen, die das Format des vom Nutzer eingegebenen Strings überprüft. Eingeben dürfen nur „0“ und „1“ – in Summe maximal 24 Zeichen. Die Helper-Funktion *format_binary* ordnet die eingegebenen Ziffern in 3er-Gruppen an, da mit der Eingabe jeweils 3 Channels von einem ADC konfiguriert werden.

```
def format_binary(P):
    P = P.replace(" ", "") # Entfernen von Leerzeichen
    # Gruppieren in 3er-Gruppen
    return ' '.join(P[i:i+3] for i in range(0, len(P), 3))

def validate_and_format_binary(P):
    # Überprüft, ob der eingegebene String nur aus 0 und 1 besteht und maximal
    24 Zeichen lang ist
    if all(c in "01" for c in P.replace(" ", "")) and len(P.replace(" ", ""))
    <= 24:
        formatted = format_binary(P) # Formatiert den String
        binary_var.set(formatted) # Setzt den Wert der Variable binary_var auf
        den formatierten String
        update_save_button_state()
        return True
    elif P == "": # Wenn das Entry-Widget leer ist, wird der Button
    deaktiviert
        update_save_button_state()
        return True
    else:
        return False
```

In einem letzten Schritt wird der Button zur Bestätigung gesteuert – in Abhängigkeit von der Eingabe des Nutzers. Der Button wird mithilfe der Funktion *update_save_button_state* erst dann aktiviert und für den User nutzbar gemacht, wenn die Zifferneingabe auch korrekt war.

```
def update_save_button_state():
    # Aktiviert den Button, wenn 24 Zeichen eingegeben wurden
    if len(binary_var.get().replace(" ", "")) == 24:
        save_binary_button.state(['!disabled'])
```

```
else:
    save_binary_button.state(['disabled'])
```

The figure shows four sequential states of a web form titled 'Quick Configuration'. Each state has a label 'ADCs & COMPs' followed by an input field and an 'Apply Quickconfig' button.

- State 1:** The input field contains the placeholder text '000 000 000 000 000 000 000'. The button is disabled (greyed out).
- State 2:** The input field is empty. The button is disabled.
- State 3:** The input field contains the text '111 110 100 011 111 111 010 1'. The button is disabled.
- State 4:** The input field contains the text '111 110 100 011 111 111 110 101'. The button is now active (blue border and text), and a mouse cursor is clicking it.

Abbildung 14 Die Steuerung des Entry-Felds und Buttons in Abhängigkeit von der Nutzereingabe: Platzhalter, geleertes Feld, unzureichende sowie vollständige Eingabe

Darüber hinaus kam das Modul *logging* zum Einsatz, um während der Laufzeit relevante Ereignisse zu protokollieren, was die Fehleranalyse und -korrektur effizienter gestaltet.

```
import logging

logging.basicConfig(filename="app.log", level=logging.ERROR)

try:
    daten = open("messwerte.txt").read()
except FileNotFoundError:
    logging.error("Die Datei wurde nicht gefunden.")
```

Der Code wurde überdies gezielt auf *Code Smells* hin überprüft und optimiert. Einige der häufigsten Code Smells, die identifiziert und korrigiert wurden, sind:

- Mehrfachverwendung von Code
- überlange Funktionen mit zu vielen Aufgaben
- unnötige Verwendung globaler Variablen

Ein Hindernis hierbei war es, die Funktionalität des Codes trotz der Umstrukturierung beizubehalten. Manches Mal waren hierfür mehrere Anläufe nötig.

Im nächsten Schritt wurde der Programmcode nach klassischen Konventionen gemäß **PEP 8** (siehe Kap. 4.4.1) vereinheitlicht und in eine strukturierte Form gebracht. Funktions-, Klassen- und Variablennamen wurden gemäß den Namenskonventionen mit aussagekräftigen Namen versehen und im `snake_case` bzw. `PascalCase` formatiert und die Verwendung von Leerzeichen und -zeilen wurde vereinheitlicht.

Für die Umsetzung dieser Formatierungsrichtlinien wurde mit dem Code-Formatter *Black* experimentiert, der den gesamten Code automatisiert einheitlich formatiert.

Ein letzter Schritt in der Finalisierung war die Erstellung einer ausführbaren Datei (**Executable**), um die Software unabhängig von einer bestehenden Python-Installation lauffähig zu machen. Hierfür kam `PyInstaller` zum Einsatz. Mit spezifischen Parametern wie `--onefile` und `--noconsole` wurde festgelegt, dass eine einzelne Datei geschrieben werden sollte, die beim Start der GUI-Anwendung kein Konsolenfenster öffnen sollte.

6 Zusammenfassung

Dieses Projekt hatte den Entwurf und die Erarbeitung einer GUI-Applikation zur Konfiguration von Messaufbauten über den CAN-Bus zum Ziel. Dabei entstand in den vergangenen Monaten eine benutzerfreundliche und effiziente Anwendung, die einen hohen Individualisierungsgrad erlaubt. Im Fokus standen dabei die zielgerichtete Softwareentwicklung sowie die Modularisierung und stetige Verbesserung des Codes. Die Anwendung sollte für den Endnutzer intuitiv zu bedienen sein, aber gleichsam stabil laufen und alle geplanten Funktionen sicher erfüllen. Während der Entwicklung taten sich zahlreiche Herausforderungen auf, die auf unterschiedliche Arten bewältigt wurden. Insbesondere die Auswahl geeigneter Technologien und die Aneignung zuvor nicht verfügbarer Kompetenzen zählen zu den Key-Learnings dieses Projekts. Dazu zählen auch die Grundsätze der GUI-Entwicklung, bei der besonders die Wichtigkeit der Validierung von Benutzereingaben zur Sicherstellung einer robusten Anwendung an dieser Stelle Erwähnung finden soll.

Die Wahl von Python 3.13 und Tkinter erwies sich als goldrichtig, da der Entwicklungsprozess von einem häufigen Rework und einer kurzfristigen Prototypentwicklung geprägt war.

Ein wesentlicher Bestandteil der Arbeit war die Code-Optimierung. Die Anwendung von unterschiedlichen Refactoring-Techniken vermittelte gleichzeitig gängige Praktiken eines guten Programmierstils. So wurden aus dem Code Redundanzen beseitigt und etwa durch Funktionen ersetzt, um die Übersichtlichkeit, Funktionalität und Wartbarkeit zu erhöhen. Try-Except-Blöcken sorgen hingegen für erhöhte Stabilität und die Beschäftigung mit dem Code-Formatter „Black“ wurde die Lesbarkeit weiter erhöht.

Trotz des erfolgreichen Abschlusses des Projekts konnte die Funktionalität in der Praxis leider nicht ausreichend erprobt werden, diese Aspekte bleiben jedoch für zukünftige Projekte zu bearbeiten. Ebenso kann die Datenvisualisierung als Desiderat für die Weiterentwicklung des Projekts gesehen werden.

Die Applikation kann weiter in puncto User Experience verbessert werden, etwa mit einer Benutzerführung, differenzierte Steuerungsmöglichkeiten (Kontextmenüs, Shortcuts etc.) und individuell konfigurierbare UI-Elemente.

Eine dezidierte Fehleranalyse (etwa durch Anwendung der Logging-Funktion) sowie die Steigerung der Performanz können die Anwendung weiter verbessern. Nachdem ein erster, funktionaler Prototyp entwickelt wurde, wären auch alternative GUI-Frameworks wie PyQt oder Kivy in Erwägung zu ziehen, da sie eine wesentlich steilere Lernkurve besitzen, aber auch mehr Möglichkeiten zur Anpassung bieten, insbesondere hinsichtlich moderner Benutzeroberflächen und erweiterter Funktionen. Die Webkonnektivität könnte noch wesentlich stärker genutzt werden, wenn es darum geht automatische Updates oder eine Anbindung an eine Web-Datenbank zum Ablegen von Konfigurationen oder Messwerten umzusetzen.

Abschließend ist zu resümieren, dass durch eine systematische Herangehensweise eine funktionsfähige Anwendung entwickelt werden konnte, die eine effiziente Konfiguration von Messaufbauten über den CAN-Bus ermöglicht. Die gewonnenen Erfahrungen und erarbeiteten Lösungen stellen für den Autor der vorliegenden Arbeit einen gewaltigen Wissens- und Kompetenzzuwachs dar, wenn es um praxisnahe

Softwareentwicklung geht. Die gewonnenen Erfahrungen bilden die Grundlage für weitere, zukünftig noch umzusetzende Projekte.

7 Literatur- und Quellenverzeichnis

- Bonacina, M. (2018). *Python 3. Programmieren für Einsteiger*. Landshut: BMU Media.
- Chandrakar, S., & Bahadure, N. (2023). *Building Modern GUIs with tkinter and Python. Building user-friendly GUI applications with ease*. London: BPB Online.
- Corrigan, S. (Mai 2016). *Introduction to the Controller Area Network (CAN)*. (T. I. Incorporated, Hrsg.) Abgerufen am 21. Februar 2025 von <https://www.ti.com/lit/an/sloa101b/sloa101b.pdf?ts=1740097187345>
- Dörn, S. (2020). *Python lernen in abgeschlossenen Lerneinheiten. Programmieren für Einsteiger mit vielen Beispielen*. Wiesbaden: Springer Fachmedien.
- Downey, A. B. (2021). *Think Python. Systematisch programmieren lernen mit Python*. Heidelberg: dpunkt.verlag.
- Electronics Stack Exchange. (1. März 2023). *STM32 SPI DMA issue*. Von Stack Exchange: <https://electronics.stackexchange.com/questions/703940/stm32-spi-dma-issue> abgerufen
- Engels, H. (2000). *CAN-Bus. Feldbusse im Überblick, CAN_Bus-Protokolle, CAN-Bus-Meßtechnik, Anwendungen*. Poing: Franzis’.
- Ernesti, J., & Kaiser, P. (2023). *Python 3. Das umfassende Handbuch*. Bonn: Rheinwerk Verlag.
- Fehr, H. (2019). *Let’s code Python*. Bonn: Rheinwerk Verlag.
- Jones, A. (2024). *Python GUI Programming with Tkinter. Developing User-Friendly Interfaces with Tkinter in Python*. Eigenverlag.
- Knoll, C., & Heedt, R. (2022). *Python für Ingenieure für Dummies*. Weinheim: Wiley-VCH GmbH.
- Matthes, E. (2020). *Python 3 Crashkurs. Eine praktische, projektbasierte Programmierereinführung*. Heidelberg: dpunkt.verlag.

- Meier, B. A. (2017). *Python GUI Programming Cookbook. Develop beautiful and powerful GUIs using the Python programming language*. Birmingham – Mumbai: Packt Publishing.
- Microchip Technology Inc. (28. Februar 2024). *MCP3461/2/4 Two/Four/Eight-Channel, 153.6 ksps, Low Noise, 16-Bit Delta-Sigma ADC Data Sheet*. Abgerufen am 26. April 2025 von Microchip Technology: <https://www.microchip.com/en-us/product/mcp3464>
- Moore, A. D. (2021). *Python GUI Programming with Tkinter. Design and build functional and user-friendly GUI applications*. Birmingham - Mumbai: Packt Publishing.
- Moore, A., & Harwani, B. (2019). *Python GUI Programming - A Complete Reference Guide. Develop responsive and powerful GUI applications with PyQt and Tkinter*. Birmingham - Mumbai: Packt Publishing.
- Paz, A. R. (2018). *Tkinter GUI Application Development Cookbook. A practical solution to your GUI development problems with Python and Tkinter*. Birmingham - Mumbai: Packt Publishing.
- Peak System Technik GmbH. (2024. September 2024). *PCAN-USB_UserMan_deu*. Abgerufen am 16. Oktober 2024 von API zur Anbindung an CAN- und CAN-FD-Busse: <https://www.peak-system.com/PCAN-Basic.239.o.html>
- Peak System Technik GmbH. (07. Februar 2024a). *PCAN - Parameters*. Abgerufen am 2024. September 2024 von CAN Interface for USB: <https://www.peak-system.com/PCAN-USB.199.o.html?L=1>
- Schäfer, C. (2019). *Schnellstart Python. Ein Einstieg ins Programmieren für MINT-Studierende*. Wiesbaden: Springer Spektrum.
- Steyer, R. (2018). *Programmierung in Python. Ein kompakter Einstieg für die Praxis*. Wiesbaden: Springer Vieweg.
- STMicroelectronics. (1. März 2024). *RM0440 Reference Manual: STM32G4 Series Advanced Arm®-based 32-bit MCUs*. (STMicroelectronics, Hrsg.) Abgerufen am 25. April 2025 von STMicroelectronics:

https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

Sweigart, A. (2020). *Routineaufgaben mit Python automatisieren. Praktische Programmierlösungen für Einsteiger*. Heidelberg: dpunkt.verlag.

Sweigart, A. (2022). *Der Weg zum Python-Profi. Ein Best-Practice-Buch für sauberes Programmieren*. Heidelberg: dpunkt.verlag.

Theis, T. (2024). *Einstieg in Python. Ideal für Programmiereinsteiger*. Bonn: Rheinwerk Computing.

Ulias, J. (2024). *Python Programming For GUI Development. A Beginner's Manual to Building Intuitive User Interfaces with Python using Tkinter, PyQt, and Beyond*. Eigenverlag.

van Rossum, G., Warsaw, B., & Coghlan, A. (01. August 2013). *PEP 8 – Style Guide for Python Code*. Abgerufen am 25. Februar 2025 von PEP 8 – Style Guide for Python Code: <https://peps.python.org/pep-0008/>

VectorNav Technologies. (8. Mai 2024). *VN-200 User Manual*. Abgerufen am 26. April 2025 von NavtechGPS: https://www.navtechgps.com/wp-content/uploads/assets/1/7/VN200UserManual_UM004_080514.pdf

Voss, W. (2017). *A Comprehensible Guide to Controller Area Network*. Auburn: Copperhill Technologies.

Weigend, M. (2019). *Python 3. Lernen und professionell anwenden. Das umfassende Praxisbuch*. Frechen: mitp Verlag.

Wikipedia. (kein Datum). *CAN-Bus*. Abgerufen am 26. April 2025 von CAN-Bus: <https://de.wikipedia.org/wiki/CAN-Bus>

Wikipedia. (kein Datum). *Ringpuffer*. Abgerufen am 26. April 2025 von Ringpuffer: <https://de.wikipedia.org/wiki/Ringpuffer>

Wouters, T. (26. Mai 2023). *PEP 719 – Python 3.13 Release Schedule*. Abgerufen am 22. Februar 2025 von <https://peps.python.org/pep-0719/>

8 Abbildungs-, Tabellen- und Abkürzungsverzeichnis

8.1 Abbildungsverzeichnis

Abbildung 1 ISO 11898 Standard Architecture (Corrigan, 2016, S. 2)	9
Abbildung 2 Bus-Topologie mit 7 Netzknoten und Abschlusswiderständen (Engels, 2000, S. 17).....	10
Abbildung 3 Veranschaulichung der Datenübertragung von den Sensoren bis zur PC- Applikation des Endnutzers	17
Abbildung 4 Vergleich zwischen einem Default-Fenster und nach ersten Anpassungen	28
Abbildung 5 Veranschaulichung der Arbeitsweise des Layout-Managers grid(); hier in einer 3x3 Geometrie	32
Abbildung 6 Eine rudimentäre Menüleiste und eine – mit den erwähnten Optionen – angepasste.....	40
Abbildung 7 Beispiele für Dialogfelder	41
Abbildung 8 Ein Fortschrittsbalken zeigt den Status eines Hintergrundprozesses an	58
Abbildung 9 Früher Entwurf des GUI-Designs in der Planungsphase	61
Abbildung 10 Die Eingabemaske zur Konfiguration des Messaufbaus	65
Abbildung 11 Die Darstellung der Widget-Hierarchie	67
Abbildung 12 Mit dem <<ListBoxSelect>>-Event wird eine Funktion zur Zustandsänderung von Buttons aufgerufen.....	70
Abbildung 13 MessageBox und Filedialog in der Anwendung.....	72
Abbildung 14 Die Steuerung des Entry-Felds und Buttons in Abhängigkeit von der Nutzereingabe: Platzhalter, geleertes Feld, unzureichende sowie vollständige Eingabe	76

8.2 Tabellenverzeichnis

Tabelle 1 Datenraten des Messaufbaus bei unterschiedlichen Abtastraten	13
Tabelle 2 Übersicht: GUI-Frameworks (Ernesti & Kaiser, 2023, S. 1228-1231).....	20
Tabelle 3 Design Patterns in der GUI-Entwicklung (Ulias, 2024, S. 103-104)	22
Tabelle 4 Technische Richtlinien guter Entwicklungspraxis (Ulias, 2024, S. 105)	23

Tabelle 5 User Experience Prinzipien (Ulias, 2024, S. 112-114) und (Jones, 2024, S. 7-8)	24
Tabelle 6 Gegenüberstellung von Layout-Managern.....	31
Tabelle 7 Exemplarische Veranschaulichung der relevanten Widget-Typen.....	37
Tabelle 8 Zeichenmethoden im Canvas (Theis, 2024, S. 388-389)	43
Tabelle 9 Frühe Beispiele für Datenvisualisierung aus der Anfangsphase des Projekts	43
Tabelle 10 Selbst gesetzte Meilensteine bei der Erarbeitung des Projekts.....	62
Tabelle 11 Grundsätzlicher CAN-Frame-Aufbau	63
Tabelle 12 Baudraten	63
Tabelle 13 Datenframes bei der Kommunikation über den CAN-Bus.....	64
Tabelle 14 Arten der Widget-Erstellung.....	66

8.3 Abkürzungsverzeichnis

ADC	Analog-Digital-Wandler
API	Application Programming Interface; Programmierschnittstelle
CAN	Controller Area Network
CAN-FD	Controller Area Network – Flexible Data Rate
GIL	Global Interpreter Lock
GUI	Graphic User Interface; Grafische Benutzeroberfläche
IDE	integrated development environment
JSON	JavaScript Object Notation
LAN	Local Area Network
MVC	Model-View-Controller
OOP	Objektorientierte Programmierung
TCL	Tool Command Language
Tkinter	Toolkit Interface
UX	User Experience